

Nativx Assistant — Audit complet de arhitectură, produs și execuție

Raport de nivel CTO / Principal Architect • 12 iunie 2026

Bază de analiză: `CLAUDE.md` (contractul de arhitectură), `Status proiect & analiză` (12.06.2026), `arhitectura_finala_productie.drawio` (C1-C3 + Deployment & Go-Live), `sales_assistant_architecture__3_.drawio` (arhitectura master + Optimization Layer P1-P9), `nativx_plan_taskuri_complet.xlsx` (193 taskuri, E0-E20, M1-M3), cardurile T001-T038 + promptul de extracție v2, plus date de piață verificate la zi (iunie 2026) — surse la final.

0. Executive summary

Verdictul scurt: ai una dintre cele mai disciplinate arhitecturi de AI assistant pe care le poate avea un proiect în faza asta — pipeline liniar cu un singur scriitor per câmp, outbox pattern, validator structural anti-halucinație, izolare multi-tenant dublă (cod + RLS) deja **testată live**, buget de context impus în cod, scară de degradare „niciodată tăcere”. Problema nu e designul. Problema e raportul **design : execuție** ≈ **85 : 15** și trei goluri strategice:

1. **„Omnichannel” există doar pe hârtie.** Obiectivul declarat e website + WhatsApp (+ alte canale), dar în cele 193 de taskuri nu există *niciun* task de web widget. Primul canal alternativ planificat e Telegram, în P2. Pipeline-ul e, din fericire, channel-agnostic prin design (`channels`, `channel_identities`, sender unic) — costul de a adăuga web e mic, dar trebuie pus în plan explicit.
2. **Termen legal dur peste ~7 săptămâni.** EU AI Act art. 50(1) — obligația de a informa utilizatorul că vorbește cu un AI — se aplică de la **2 august 2026**, cu amenzi de până la 15 mil. € sau 3% din cifra de afaceri globală. Taskul T180 („notă informare în primul mesaj”) există, dar e parcat în P1-O2. Trebuie mutat în categoria *launch-blocking*: niciun client live fără el.
3. **Infrastructura de producție e un singur VPS cu docker-compose, un singur Redis, Supabase Free.** Pentru demo și clientul 1 e corect (nu supra-construi). Pentru SLO-urile pe care chiar tu le-ai scris în diagrama de Deployment (RTO 1h / RPO 5min, sondă la 5 min, p95 SLO) e insuficient — diagrama promite mai mult decât livrează planul de taskuri.

Contextul de piață 2026 îți validează teza. eMAG a lansat oficial iZi în martie 2026 (după beta din februarie), cu metrice publice bune (2,6 întrebări/conversație, 1 din 3 utilizatori revin săptămâna următoare, >90% evaluări pozitive) — conversational commerce e acum un comportament educat pe piața din România, dar iZi e *first-party*, doar pentru eMAG.

Restul retailului românesc nu are așa ceva, iar competitorii globali (Intercom Fin la \$0,99/rezoluție, Gorgias la \$0,90-1,00/rezoluție + taxe pe tichete) sunt scumpi, support-first și slabi pe WhatsApp-commerce în limba română. Fereastra ta există. E reală. Dar e o fereastră, nu o ușă — Hornbach România deja a lansat asistent virtual AI în bricolaj, deci incumbenții se mișcă.

Economia unitară e arma ta secretă. Cu GPT-5.4-mini/nano (lansate în martie 2026: mini \$0,75/\$4,50 per 1M tokeni, nano \$0,20/\$1,25, input cache-uit la 10% din preț, Batch –50%) plus straturile gratuite țintite la 40-60% din trafic, costul tău LLM per conversație iese realist la **\$0,01-0,03**. Competiția vinde rezoluția cu \$0,90-0,99. Ai două ordine de mărime de marjă în care să te joci cu pricingul de agenție.

Scoruri (detaliate în §11)

Dimensiune	Scor	Pe scurt
Arhitectură	8 / 10	Design excepțional pentru stadiu; minus pentru SPOF-uri (VPS unic, Redis unic), componente desenate dar fără taskuri (WAF/gateway, model gateway, CDC) și faptul că 7 din 9 stagii nu există încă în cod.
Scalabilitate	7 / 10	Solid până la ~1.000 de tenanți cu upgrade-uri operaționale previzibile; partiționare, RLS, pgvector per-tenant și outbox sunt alegeri care îmbătrânesc bine. Peste 1.000: Redis HA, autoscale workers, ClickHouse. Peste 5-10.000: arhitectură celulară.
Produs	6 / 10	Wedge clar (WhatsApp sales pentru retail RO, serviciu managed), buclă de bani gândită (atribuire ref → orders → dashboard). Minus: un singur canal real, zero self-serve, dashboard client în P2, fără logică de discount, fără CRM.
AI capabilities	6,5 / 10	Triaj+agent+validator+caching = arhitectură LLM matură; memorie pe 3 niveluri (state, sumar, profil) bine gândită. Minus: embeddings = 0 azi, eval pipeline în P2, fără motor de recomandare dincolo de retrieval, vision/STT în P2.

Top 5 acțiuni imediate (restul în Top 20, §12)

1. **T017 azi** (cheie OpenAI + limite spend) — e pe drumul critic a tot ce înseamnă LLM și embeddings; fără el M2 nu există.
2. **Mută T180 (disclosure AI) în MVP** și adaugă-l în go-live checklist ca item legal, nu de curtoazie — deadline 2 august 2026.
3. **Pune Cloudflare (free) în fața VPS-ului** înainte de orice trafic real: WAF + rate-limit global + TLS + ascunderea IP-ului. Diagrama ta master desenează „WAF + API

Gateway" — asta e versiunea de 2 ore a acelei căsuțe.

4. **Decide web widget-ul acum** ca epic V1.5 (după clientul 1 pe WhatsApp): aceeași conductă, un adapter de canal nou + un serviciu SSE/WebSocket. E diferența dintre „bot de WhatsApp” și produsul omnichannel pe care îl vinzi în pitch.
 5. **Regenerează cardurile T021-T034** pe schema reală (plată, `business_id`) sau marchează-le vizibil OBSOLETE în zip/repo — azi ele contrazic `schema_reference.md` și vor otrăvi sesiunile Claude Code care le citesc.
-

Cuprins

1. Partea 1 — Audit complet pe componente
 2. Partea 2 — Analiza arhitecturii și scalare 100 / 1.000 / 10.000
 3. Partea 3 — Viziunea produsului și competiția 2026
 4. Partea 4 — Roadmap funcțional MVP / V1 / V2 / V3
 5. Partea 5 — Capabilități de Sales Assistant AI
 6. Partea 6 — Multi-tenant SaaS
 7. Partea 7 — WhatsApp + Web Widget
 8. Partea 8 — Audit de securitate
 9. Partea 9 — Backlog nou + corecții la planul existent
 10. Partea 10 — Arhitectura finală pe 3 ani
 11. Scoruri detaliate
 12. Top 20 priorități
 13. Roadmap 6 luni / 12 luni
 14. Surse 2026
-

PARTEA 1 — Audit complet pe componente

Format per componentă: ☒ Bine · ☐ Greșit/șubred · ☐ Lipsește · ☐ Risc · ☐ Impact la scalare.

1.1 Structura proiectului & organizarea codului

☒ **Bine.** Structura `(src/)` oglindește exact pipeline-ul (webhook/, worker/stages/, tools/, agent/, proactive/, jobs/, gdpr/) — arhitectura se citește din filesystem, ceea ce e rar. Config disciplinat (totul prin `(Settings)`, nimic din `(os.environ)`), `(TurnContext)` cu owner documentat per câmp, helperi `(emit())`/`(set_reply())` care țin stagiile ignorante față de

măsurare (principiul 10). CI ca poartă (ruff + pytest required), CODEOWNERS pe `prompts/` și `docs/*.sql` — exact fișierele unde o greșeală costă bani.

 **Greșit/șubred.** Cardurile T021-T034 din zip referențiază schema veche cu 4 scheme (`conv.`, `core.`, `catalog.`) — de exemplu T030 scrie `conv.gdpr_requests`, `core.audit_log`, `orders.customer_phone`, toate inexistente în schema_v2 (telefonul nici nu mai stă pe orders, corect). Documentul de status le declară obsolete, dar fișierele trăiesc nealterate și „promptul de extracție v2” cere explicit ca noile carduri să copieze stilul celor existente — adică să copieze și numele greșite. Pentru un workflow „o sesiune Claude Code = un card”, asta e contaminare directă de context.

 **Lipsește.** ADR-uri (architecture decision records) — decizia majoră „schemă plată în loc de 4 scheme” e documentată doar în `schema_reference.md`; merita un ADR scurt cu motivație, ca să nu se redeschidă discuția peste 6 luni. Un `make bootstrap` care validează tot lanțul (env → DB → Redis → ping OpenAI) pentru onboarding.

 **Risc.** Procesul git a pierdut deja de 2 ori commit-uri orfane (notat onest în status). Cu 2 oameni e recuperabil; cu 4+ devine pierdere de muncă reală. Regula propusă (1 PR = 1 commit logic complet) e bună — pune-o în CONTRIBUTING ca regulă, nu ca recomandare.

 **Impact la scalare.** Pozitiv: structura per-stagiu permite extragerea în servicii separate fără rescriere (vezi Partea 2). Negativ: niciun mecanism de versionare a configului per tenant (azi `businesses.settings`) jsonb fără schemă/migrare) — la 50+ clienți, schimbările de format în `settings` devin migrare manuală.

1.2 Arhitectura pipeline (cele 9 stagii)

 **Bine.** Pipeline liniar fără bucle de orchestrare, LLM doar în 2 puncte, early-exit la orice stadiu, un singur punct de ieșire (Sender → outbox → dispatcher). Asta elimină prin construcție clasele de bug-uri care omoară majoritatea boților: race conditions pe conversație (lock + FIFO per stream), mesaje duble (dedupe + idempotency_key), tăcerea la eroare (scara de degradare), state umflat (8KB tăiat în cod + CHECK în DB). Debounce-ul adaptiv 2-3s cu *lot de mesaje, nu string lipit* arată înțelegerea reală a comportamentului utilizatorilor de WhatsApp. Prefetch în fereastra de debounce (context + embedding în paralel) e un detaliu de senior.

 **Greșit/șubred.** Stadiu real: din 9 stagii, 1 e parțial (webhook GET), 0 complet end-to-end. `search_products` există doar pe filtre SQL (embeddings = 0). Adică tot ce e diferențiator (triaj, agent, validator, sender tranzacțional) e ne-testat în realitate. Nu e o critică a designului — e un memento că scorul de arhitectură de mai jos plătește pentru hârtie, nu pentru producție.

 **Lipsește.** Definiția formală a comportamentului la *mesaje primite în timpul procesării* (utilizatorul mai scrie ceva cât agentul lucrează): lock-ul serializează, dar lotul următor vede răspunsul deja trimis? Recomand: după send, dacă stream-ul are mesaje noi acumulate,

turul următor primește în context și replica proprie tocmai trimisă (o ai din messages) — altfel botul răspunde la întrebarea veche.

 **Risc.** MAX 3 tool calls / tur e sănătos pentru cost, dar va tăia fluxuri legitime (search → details → compare → checkout = 4). Soluția nu e creșterea limitei, ci continuarea în turul următor cu state („ți-am pregătit comparația, vrei link de comandă?") — documentează-o ca pattern în promptul agentului, altfel agentul va încerca să comprime și calitatea scade.

 **Impact la scalare.** Excelent: fiecare stagiou e funcție pură testabilă, runner-ul măsoară din exterior; orizontal scaling = mai mulți consumatori pe consumer group, lock-ul per conversație păstrează corectitudinea. Singura buclă de creștere ascunsă: `semantic_cache` cere un embedding per mesaj la lookup (apel OpenAI, ~50-150ms + cost mic) — la volum mare, pune un cache exact-match Redis pe `phrase_norm` ÎNAINTE de embedding (aliasele fac deja asta parțial).

1.3 Design patterns

 **Bine.** Outbox pattern (scriere tranzacțională reply+state+message, dispatcher idempotent cu retry/backoff/dead) — pattern-ul corect, implementat la nivel de schemă deja (`outbox.idempotency_key UNIQUE`), `FOR UPDATE SKIP LOCKED` planificat în G1). Optimistic locking pe state (`state_version`). Single-writer per câmp pe `TurnContext`. Identity map prin `channel_identities` cu `external_id_hash` generat. State = ref-uri, nu obiecte. Toate sunt alegeri de manual, aplicate consecvent.

 **Greșit/șubred.** „Promptul se generează din DB" (principiul 9) e corect, dar sursa s-a subțiat: planul inițial avea `taxonomy`; schema_v2 are doar `categories` + `intent_aliases`. Pentru beauty (vertical cu *concerns*: ten gras, sensibil, anti-aging), filtrarea pe `attributes jsonb` fără taxonomie controlată înseamnă că triajul (care emite `filters`) și `search_products` (care le mapează pe WHERE) trebuie să împartă un vocabular — azi acel vocabular nu are sursă unică. Decizia din schema_reference („taxonomy se adaugă aditiv când verticalul cere") e OK, dar verticalul tău demo *deja cere*.

 **Lipsește.** Circuit breaker formal în LLM adapter (e desenat în diagramă la Model Gateway, nu există task — T101 acoperă doar retry + usage). Saga/compensare pentru `book_appointment` (scrii în DB + Google Calendar = două sisteme; ce faci când al doilea pică?).

 **Risc.** Mic, dar real: `clarification_templates` nu există în schema_v2, clarificările „vin din cod/prompt" — asta sparge principiul 9 exact pe fluxul cu cea mai mare frecvență (CLARIFY e ruta default la incertitudine). Ori adaugi tabelul (5 rânduri seed per limbă, T036 era deja scris), ori accepți hardcodare și o notezi ca excepție explicită.

 **Impact la scalare.** Patterns-urile alese sunt exact cele care permit trecerea monolit → servicii fără rescriere: outbox-ul devine graniță naturală spre un dispatcher separat, streams-urile devin Kafka topics, stagiile devin pods. Nota maximă aici.

✅ **Bine.** Alegerea modelelor e la zi și corectă economic: GPT-5.4-mini pentru vânzare (\$0,75/\$4,50 per 1M, cache input \$0,075), GPT-5.4-nano pentru triaj/simple/extracții (\$0,20/\$1,25, cache \$0,02), ambele cu context 400K și Batch –50% pentru joburi offline (review summaries, LLM-as-judge). Prefixul static byte-identic pentru prompt caching e gândit de la început (T086 are chiar DoD pe `cached_tokens > 0`). Triaj cu output JSON validat Pydantic + „categoria inventată → CLARIFY” e apărarea corectă contra hallucinated routing. Validatorul structural (preț/produs/link contra retrieval, 1 retry, apoi formulare fără cifre) e cea mai valoroasă piesă de încredere din tot sistemul — competitorii o vând ca feature premium.

⚠️ **Greșit/șubred.** Dependență de un singur provider, asumată („failover provider LLM” e P4 în Optimization Layer, fără task). Cu scara de degradare mini→nano→template ai *availability*, dar un incident OpenAI de ore îți pune toți clienții pe template. Pentru un produs care promite vânzare, acceptabil în primele 3 luni, nu după.

❑ **Lipsește.** (1) Moderation pe inbound — endpoint-ul de moderare OpenAI e gratuit; un gate ieftin înaintea agentului taie jailbreak-urile evidente și conținutul toxic înainte să consume mini. (2) Politică de *data retention* la OpenAI: pentru clienți GDPR-sensibili, activează regional processing / ZDR unde e disponibil (atenție: endpoint-urile cu rezidență regională au uplift de preț ~10% la nano — decizie comercială, documentează-o). (3) Versionarea prompturilor e P1 (T132) — adu măcar hash-ul promptului în `analytics_events` din prima zi de agent, costă 3 linii.

🔥 **Risc.** `ai_summary` e azi fictiv/templat pe datele demo; agentul de vânzare va suna generic exact în demo-urile de vânzare a serviciului. Pentru clientul 1 real, jobul de enrichment (feed → ai_summary prin LLM, Batch) trebuie tratat ca parte din onboarding, nu ca „nice to have” — calitatea copy-ului per produs E produsul.

☑️ **Impact la scalare.** Costul crește liniar cu mesajele care trec de straturile gratuite; pârghiile sunt deja în design (cache semantic, aliase, prompt caching, nano pentru simple). Singura neliniaritate: re-embed la sync-uri mari de catalog — `content_hash` o rezolvă (re-embed doar la schimbare reală), păstrează-o sfântă.

1.5 Baza de date

✅ **Bine.** Schema_v2 e matură: 49 tabele, partiționare lunară pe `messages` și `analytics_events`, pgvector cu HNSW + `content_hash`, idempotență prin chei unice (`business_id`, `provider/external_id`), PII concentrat într-un singur tabel (`channel_identities`) cu hash generat, `in_24h_window()` ca funcție derivată (nu flag stocat — corect, flag-urile mint), `gdpr_erase_contact()` security definer cu anonimizare-nu-ștergere (analytics agregat rămâne valid). RLS pe toate tabelele + rol `bot_runtime` fără `bypassrls` + `SET app.business_id` per conexiune — și, crucial, **testat live**: query fără filtru → 0 rânduri. Asta e diferența dintre „avem RLS” și „avem izolare”.

⚠ **Greșit/șubred.** (1) `(product_url)` NULL pe toate cele 500 de produse demo → `(checkout_link)` n-are bază, validatorul de linkuri n-are ce valida, iar demo-ul de vânzare se termină fără link de cumpărare — exact pasul care face banii. (2) `(statement_cache_size=0)` pe pooler (corect pentru pgbouncer transaction-mode, dar lasă performanță pe masă) — la migrarea pe conexiune directă/Supervisor session-mode, revaluează. (3) Conectarea ca `(postgres)` + `(SET ROLE bot_runtime)` prin pooler funcționează, dar înseamnă că o eroare de cod care sare peste SET ROLE rulează ca superuser — adaugă un assert la checkout din pool `((SELECT current_user))` în dev/staging.

❑ **Lipsește.** (1) Automatizarea partițiilor: cine creează partiția lunii viitoare? `(cleanup.py)` șterge partiții vechi, dar nu există job de *create* — folosește `pg_partman` sau un `pg_cron` cu `(CREATE TABLE ... PARTITION OF)` cu 2 luni înainte; o partiție lipsă = INSERT-uri picate = webhook-uri pierdute. (2) Backup real: Supabase Free nu are PITR; T018 notează „Pro înainte de go-live” — fă din asta un gate hard în go-live checklist, plus T162 (restore test) rămâne obligatoriu. (3) Indexuri de verificat la implementare: `(conversations(business_id, contact_id, status))` pentru load, `(outbox(status, next_attempt_at))` partial pe pending — sunt în T032, doar confirmă că au supraviețuit reconcilierii schemei.

🔥 **Risc.** HNSW pe `(product_embeddings)` e index global pe tabel multi-tenant; căutarea „filtre SQL dure apoi ORDER BY embedding <=>” e corectă, dar atenție la planul real: la filtre foarte selective Postgres poate alege scan secvențial pe subset (bine), la filtre largi poate folosi HNSW global și filtra după (recall per-tenant poate suferi). Mitigare pragmatică: `(EXPLAIN ANALYZE)` pe queries reale per tenant mare + `(hnsw.ef_search)` ajustabil; la zeci de tenanți mari, ia în calcul partiționarea `(product_embeddings)` pe `hash(business_id)`.

📈 **Impact la scalare.** Foarte bun până la ~1.000 tenanți (vezi Partea 2). Punctele de cotitură: `analytics_events` ca tabel OLTP (mută rapoartele pe `usage_daily` — deja regula există — și mai târziu CDC → ClickHouse), și conexiunile prin pooler la zeci de workeri.

1.6 API-urile (webhook + interfețe interne)

✅ **Bine.** GET /webhook (verify Meta) implementat + testat. Contractele interne (`RouteDecision`, `Reply`, `ToolResult`, `InboundMessage`) sunt Pydantic strict — JSON invalid de la LLM ridică `ValidationError`, nu crash (T042). Handler global care întoarce 200 către Meta chiar și la eroare internă (T056) — corect, altfel retry-storm.

⚠ **Greșit/șubred.** POST /webhook nu există încă (faza următoare, normal). Webhook-ul de comenzi (`(/orders)`) e „stub generic format propriu” — suficient pentru atribuire simulată, dar fiecare platformă reală (Shopify/Woo/Magento/feed custom) are semnături și payload-uri proprii; T210 (discovery platformă client 1) e corect marcat ca blocant.

❑ **Lipsește.** Un API de administrare minim (chiar și intern, protejat): toggle `bot_active`, trigger `gdpr_erase` (T177 îl are), re-sync catalog, replay outbox dead — azi toate ar fi `(psql)` la

mână. Și: rate-limit la nivel de endpoint pe /webhook (Meta e sursa, dar oricine cu URL-ul poate face POST — semnătura te apără de procesare, nu de CPU).

🔥 **Risc.** Niciun versioning pe payload-urile interne din Redis stream — dacă schimbi formatul InboundMessage cu mesaje în zbor în stream, consumerul nou pică pe ele. Pune `schema_version` în payload de la prima implementare (T049), costă nimic.

☑ **Impact la scalare.** ACK < 50ms cu scrieri fire-and-forget e designul corect pentru burst-uri; la volum, webhook-ul devine I/O-bound pe Redis — XADD e ieftin, ține dedupe-ul pe INSERT ... ON CONFLICT cât mai devreme și restul async.

1.7 Integrarea WhatsApp (Meta Cloud API direct)

✅ **Bine.** Direct pe Cloud API, fără Twilio — elimini \$0,005/mesaj markup BSP și o dependență. Semnătură X-Hub-Signature-256 cu comparație constant-time, dedupe pe provider_msg_id, `last_inbound_at` pentru fereastra de 24h, statusuri delivered/read/failed în `message_status_events`, template lifecycle complet (draft→submitted→approved→paused→deprecated, cu `provider_template_id`), proactiv DOAR cu consent + (fereastră 24h SAU template approved), opt-out STOP (T206), typing indicator, split natural < 2 mesaje. Modelul tău mental al platformei e corect și la zi: din iulie 2025 Meta taxează **per mesaj template livrat** (nu per conversație), mesajele service în fereastră deschisă sunt gratuite, iar template-urile utility trimise în interiorul ferestrei deschise sunt și ele gratuite — arhitectura ta exploatează exact regulile astea.

⚠ **Greșit/șubred.** Nimic structural. Două nuanțe de cost/operare: (1) sub pricing-ul per-mesaj, fiecare template proactiv e o taxă separată — follow-up de coș + AWB + back-in-stock către același contact în aceeași zi = 3 taxe dacă fereastră e închisă; pune costul de template în `usage_daily.templates_sent` × tarif (există câmpul) și arată-l clientului. (2) Meta poate actualiza tarifele trimestrial (1 ian/apr/iul/oct) — abonează-te la changelog, nu hardcode tarife.

❑ **Lipsește.** (1) **Monitorizarea quality rating + messaging limits per număr:** numerele noi pornesc cu limită mică de conversații business-initiated/24h și urcă pe tier-uri pe baza calității; un client cu campanii agresive își poate degrada numărul — citește `phone_number_quality_update` din webhooks și alertează. (2) **Click-to-WhatsApp ads:** mesajele inițiate din reclame CTWA deschid fereastră gratuită extinsă (72h, inclusiv template-uri) și vin cu `referral` payload — e și canal de achiziție pentru clienții tăi, și sursă de atribuire (referral → analytics_events); azi nu e nicăieri în plan. (3) Mesaje interactive native (liste, butoane, product messages din catalogul Meta) — parserul T062 le citește inbound; outbound folosești text simplu. Listele/butoanele cresc conversia pe mobil; product messages cer sync de catalog la Meta — evaluează în V2.

🔥 **Risc.** Verificarea Meta Business (T016) e 3–15 zile și e blocant extern pentru numărul de producție — e corect pornită din ziua 1; ține un buffer în promisiunile comerciale către clientul 1.

☑ **Impact la scalare.** Per-client număr propriu ⇒ throughput-ul Meta per număr nu e bottleneck-ul platformei tale; dispatcher-ul tău e. Dimensionează dispatcherul pe (mesaje/s per tenant) cu fairness — un broadcast back-in-stock al unui client mare nu are voie să întârzie răspunsurile conversaționale ale celorlalți (două cozi: conversational cu prioritate, proactive cu pacing).

1.8 Website widget

☑ **Bine.** Singurul lucru bun: arhitectura nu îl blochează. `channels.kind`, `channel_identities`, sender unic și pipeline channel-agnostic înseamnă că widgetul e un adapter, nu o rescriere.

⚠ **Greșit/șubred.** Nu există — nici task, nici schiță. În ambele diagrame canalele sunt „WhatsApp / Telegram”. Obiectivul declarat al produsului („pe website, pe WhatsApp...”) nu are acoperire în plan.

☐ **Lipsește.** Tot: embed script, serviciu de sesiuni web (SSE/WebSocket), identitate vizitator anonim → merge în contact la capturarea telefonului/emailului, UI de chat, rate-limit pe origin, CORS/CSP. Detaliat în Partea 7 + backlog (Partea 9).

🔥 **Risc.** De poziționare: fără widget, „omnichannel AI Sales Assistant” e în realitate „WhatsApp bot” — iar pitch-ul tău de diferențiere față de tool-urile de WhatsApp marketing (de care România e plină) slăbește. Riscul invers există și el: să-l construiești prea devreme și să întârzie clientul 1. Recomandarea mea fermă: clientul 1 pe WhatsApp întâi, widget ca epic imediat după stabilizare (luna 3-4), pe aceeași conductă.

☑ **Impact la scalare.** Pozitiv: web-ul n-are fereastra de 24h și nici taxe per mesaj — costul marginal e doar LLM; e canalul unde marja ta e maximă și unde demo-ul e instant (un script tag la client vs. verificare Meta de 2 săptămâni).

1.9 Autentificare & autorizare

☑ **Bine.** Separarea rolurilor DB e exemplară: `bot_runtime` (RLS, fără bypass), `service_role` doar migrări/admin, `gdpr_svc` doar pentru funcțiile definer. Dashboard pe `auth.uid()` → membership în `business_users`. Secretele canalelor prin `credentials_ref` (referință spre secret manager, nu valori în DB).

⚠ **Greșit/șubred.** `business_users.role` există, dar nu e definit un model de roluri (owner/agent/viewer?) și nicio politică RLS nu diferențiază pe rol — pentru inbox handoff (P2) vei avea nevoie: agentul vede conversații, nu setări de billing.

☐ **Lipsește.** (1) Care secret manager, concret? Pe stack-ul actual: Supabase Vault e suficient și aproape gratuit ca efort; alternativ Doppler/Infisical. Decide și scrie în ADR. (2) Autentificarea endpoint-urilor admin (T177 „endpoint protejat” — protejat cum? recomand token serviciu + IP allowlist la început). (3) Rotația token-urilor Meta (system user token cu expirare lungă, dar planifică rotația).

🔥 **Risc.** Scăzut acum (nu există dashboard). Crește brusc la T221 (inbox) — acolo intră primii utilizatori externi în sistemul tău.

☑️ **Impact la scalare.** Modelul Supabase Auth + business_users ține fără probleme la mii de utilizatori de dashboard; enterprise (V3) va cere SSO/SAML — nu construi nimic acum, doar nu lega logica de auth de email/parolă hardcodat.

1.10 Observabilitate, logging, monitoring

✅ **Bine.** Observabilitate „din runner” (stagiile nu știu că sunt măsurate), trace_id per tur propagat (T053), PII redaction în logger cu test dedicat (T054), Sentry cu business_id+trace_id fără PII (T165), sondă sintetică e2e la 5 min cu alertă la 2 eșecuri (T168) — sonda e genul de lucru pe care echipele îl adaugă după primul incident; tu îl ai în plan înainte. Query-uri standard de cost/latență (T166) + alerting minim pe erori repetate / cost spike >2× / dead letters (T167).

⚠️ **Greșit/șubred.** Diagrama promite „OTel tracing per tur · SLO p95 · status page”; planul livrează loguri JSON + Sentry + SQL-uri manuale. E gapul corect pentru MVP, dar recunoaște-l: nu ai percentile reale de latență per stagiou până nu le agregi (analytics_events.latency_ms există — un query p95 pe el e suficient la început).

❑ **Lipsește.** (1) Alertă pe **lag-ul consumer group-ului Redis** (XINFO GROUPS → pending/lag) — e semnalul cel mai timpuriu că workerii nu fac față; niciun task nu-l acoperă. (2) Alertă pe **adâncimea outbox pending** și pe vârsta celui mai vechi pending. (3) Dead-letter *queue vizibilă* + procedură de replay (DLQ e desenat în diagramă; în plan există doar status `dead` pe outbox — suficient, dar scrie runbook-ul de replay).

🔥 **Risc.** Fără metricele de coadă (lag, pending), primul tău semnal de saturație va fi sonda sintetică — adică după ce clienții deja așteaptă. Cele două alerte de mai sus sunt 2-3 ore de muncă.

☑️ **Impact la scalare.** analytics_events partiționat + rollup nocturn în usage_daily e modelul corect; dashboardul și facturarea citesc DOAR din usage_daily (regulă explicită — excelent). La >1.000 tenanți, mută explorarea ad-hoc pe ClickHouse prin CDC, nu pe OLTP.

1.11 Analytics & atribuire

✅ **Bine.** Modelul generic de evenimente (intent_detected/route/tool_call/cache_hit/handoff), append-only cu bot doar INSERT, rollup idempotent (T195 cu DoD pe re-ulare = aceleași cifre), și mai ales **bucla de bani**: checkout_links cu ref_code → webhook orders → attribution (none/assisted/direct_bot) → revenue_attributed în usage_daily → raport lunar per client (T196). „Dashboard revenue per client — retainerul se vinde singur” din Optimization Layer e exact teza comercială corectă.

⚠️ **Greșit/șubred.** Atribuirea e doar last-click pe ref_code; un client va întreba „dar conversațiile care n-au dat click pe link și totuși au cumpărat?”. Modelul `assisted` există în

enum — definește-l: comandă în ≤ 7 zile de la o conversație cu produsul respectiv afișat = assisted. Fără definiție scrisă, cifra din raport devine subiect de negociere la fiecare factură.

☐ **Lipsește.** (1) Funnel explicit (conversație → produs afișat → link → click → comandă) — toate evenimentele există, lipsesc 2 query-uri și un grafic Metabase. (2) Cohorte de revenire (1 din 3 utilizatori revin săptămâna următoare e metrica cu care iZi și-a făcut PR-ul — măsoar-o și tu, e un SELECT). (3) A/B pe prompturi e P2 (canary T230) — ok ca timing.

 **Risc.** Click tracking (T191, redirect /r/{ref}) introduce un hop pe domeniul tău în linkul de cumpărare — dacă domeniul tău e blocat/încet, pierzi conversația clientului. Ține redirectul pe subdomeniul clientului unde se poate (CNAME), sau măcar pe un domeniu „curat” dedicat, nu pe cel de API.

☒ **Impact la scalare.** usage_daily ca singură sursă pentru facturare = decizia care îți permite să schimbi tot dedesubt fără să atingi banii. Păstrează-o.

1.12 Deployment & infrastructură

☒ **Bine.** Pentru faza actuală: docker-compose dev complet, Dockerfile multi-stage non-root <300MB, VPS hardening (T155: ufw, fail2ban, ssh key-only, unattended-upgrades), staging+prod separate pe același VPS, Caddy TLS auto, CD pe staging la merge + prod cu approve manual, rollback pe tag testat (T161), restore test din PITR (T162). Filosofia din diagramă — „compose la start → K8s la scară” — e matură: nu plătești taxa K8s înainte să ai trafic.

 **Greșit/șubred.** Un VPS = un singur punct de eșec pentru webhook + worker + dispatcher + scheduler + Redis simultan. Meta retrimite webhook-urile la timeout (te salvează pe ferestre scurte), dar Redis pe același host înseamnă că un disc plin omoară și coada, și aplicația. RPO 5min/RTO 1h din diagramă nu sunt atinse de planul actual pentru Redis (e declarat „doar efemer, rebuild ok” — adevărat pentru lock/debounce, FALS pentru stream-urile cu mesaje neprocesate: alea sunt date în zbor).

☐ **Lipsește.** (1) AOF/RDB pe Redis (appendonly yes, everysec) — 1 linie de config care transformă „pierdem mesajele în zbor la crash” în „pierdem $\leq 1s$ ”. (2) Cloudflare în față (WAF/rate-limit/ascundere IP) — vezi acțiunea imediată #3. (3) Healthcheck-uri compose cu restart policy + `docker compose up` la boot (systemd unit). (4) Un al doilea VPS ieftin pentru staging separat fizic — opțional, dar ieftin.

 **Risc.** Build-ul Docker e „de verificat pe VPS — fără Docker local” (status doc). Verifică-l înainte de Z11, nu în Z11.

☒ **Impact la scalare.** Drumul e clar și ieftin: VPS vertical → al 2-lea VPS (webhook+dispatcher pe unul, workers pe altul, Redis managed) → K8s/HPA. Nimic din design nu te încuie.

1.13 Calitatea planului de execuție (xlsx + carduri)

✅ **Bine.** 193 de taskuri cu DoD verificabil pe fiecare, dependențe explicite, ore estimate, milestones cu definiții operaționale (M1 echo e2e ziua 5, M2 conversație de vânzare ziua 10, M3 prod live ziua 14), reguli de tăiere („NU se taie niciodată: validator, outbox, dedupe, golden tests” — exact prioritățile corecte), blocaje externe identificate (Meta verification, platforma clientului 1) cu pornire din ziua 1. Cardurile detaliate (T030 e un exemplu de card excelent: ordine corectă a operațiilor în funcția GDPR, test pe contact sintetic, grant-uri). Capacitatea e calculată onest (S 122h utile / 2 săpt, J 52h).

⚠️ **Greșit/șubred.** (1) Cardurile T021-T034 contrazic schema reală (vezi 1.1). (2) Bugetul MVP e la limită prin construcție: 125,5h Senior planificate vs ~122h disponibile — zero buffer pentru necunoscute, iar primele 2 zile au consumat deja (corect) timp pe reconcilierea schemei. Realist: M2 alunecă 2-3 zile; planifică-o ca pe o știre, nu ca pe un eșec. (3) T147 (load test 50 conversații) e P1-O1 — bine; dar chaos T149 (kill worker mid-turn) merită mutat lipit de T146, e ieftin și validează exact garanțiile pe care le vinzi.

❑ **Lipsește din plan (acoperit în Partea 9).** Web widget; AI Act disclosure ca launch-blocking; Cloudflare/edge; monitor quality rating WhatsApp; CTWA referral; Redis persistence; alerte lag/outbox; regenerarea cardurilor obsolete; discount/promo logic; export CRM.

🔥 **Risc.** Modelul „o sesiune Claude Code = un task” e bun, dar promptul de extracție v2 trimite explicit la cardurile vechi ca gold standard — actualizează promptul să trimită la `schema_reference.md` ca autoritate de nume ÎNAINTE de orice generare nouă.

✅ **Impact la scalare.** Procesul (CI gates, golden obligatoriu la schimbări de prompt, CODEOWNERS pe SQL/prompts) e deja construit pentru o echipă mai mare decât aveți — asta e un compliment.

PARTEA 2 — Analiza arhitecturii

2.1 Diagrama conceptuală a arhitecturii ACTUALE (ce există + ce e contractat în cod/schemă)

Legendă:  implementat & testat ·  parțial ·  doar design (schemă/diagramă/task)



Scară	Volum estimat	Verdict	Ce trebuie schimbat
100 clienți	~30-80k msg în/zi · vârf 3-8 msg/s · ~1,5-4k conv/zi · LLM ~0,5-2 req/s	DA, cu arhitectura actuală.	VPS bun (8 vCPU/16GB) + Redis AOF + Supabase Pro + Cloudflare. 2-4 procese worker. Nimic exotic.
1.000 clienți	~0,3-0,8M msg în/zi · vârf 30-80 msg/s · LLM 5-20 req/s · ~10-25M rânduri/lună în messages	DA, cu upgrade-uri operaționale, fără rescriere.	Workers pe noduri separate cu autoscale (K8s sau echivalent), Redis managed cu HA, dispatcher scalat orizontal (SKIP LOCKED permite N instanțe), read-replica pt. dashboard/Metabase, CDC→ClickHouse pt. analytics ad-hoc, model gateway cu failover. Postgres: partițiile + indexurile compuse țin; monitorizează poolerul.
10.000 clienți	~3-8M msg în/zi · vârf 300-800 msg/s · LLM 50-200 req/s · ~100-250M rânduri/lună	NU în forma actuală — dar evoluția e trasată, nu e rescriere.	Arhitectură celulară: N celule independente a câte ~500-1.500 tenanți (fiecare celulă = Postgres + Redis/Kafka + workers proprii), un control-plane comun (onboarding, billing, registry tenant→celulă). Kafka în locul Redis Streams în interiorul celulelor mari. Embeddings/vector: rămâi pe pgvector per celulă SAU Qdrant per celulă dacă recall/latency cer. Multi-regiune doar dacă ieși din RO/UE.

Punctul important: **deciziile deja luate (business_id pe tot, outbox, partiționare, sender unic, stagii pure) sunt exact cele care fac drumul ăsta incremental.** Majoritatea proiectelor trebuie să rescrie ca să ajungă la 1.000; tu trebuie doar să operezi.

2.3 Bottleneck-urile, în ordinea în care le vei lovi

1. **Cheia OpenAI lipsă (azi).** Nu e glumă: e singurul blocant absolut al M2. T017 = 30 de minute.
2. **VPS-ul unic** — primul incident real (disc plin, kernel update, OOM) oprește tot. Lovit: oricând. Mitigare ieftină: alerting + Redis AOF + restart policies; mitigarea reală: split pe 2 noduri la primii 10 clienți plătitori.
3. **Redis fără persistență** — mesaje în zbor pierdute la crash. Lovit: primul crash. Fix: 1 linie (appendonly) + alertă lag.
4. **Lag consumer / un singur proces worker** — debounce 2-3s + LLM 2-5s ⇒ un worker procesează ~0,2-0,4 tururi/s; la 100 clienți activi simultan ai nevoie de 4-8 workeri.

Lovit: prima campanie proactivă a unui client. Fix: consumer group cu N workeri (designul îl permite deja, T049).

5. **Dispatcher single-instance** — outbox-ul se golește serial. Fix: N dispatchere cu FOR UPDATE SKIP LOCKED + fairness conversational > proactiv.
6. **Poolerul Supabase (transaction mode)** — `statement_cache_size=0` + SET ROLE per checkout; la zeci de conexiuni concurente apar așteptări. Lovit: ~300-500 clienți sau dashboard intens. Fix: Supervisor session pools dedicate per proces / conexiune directă pentru workeri + pgbouncer propriu.
7. **analytics_events ca destinație de query-uri** — dacă cineva face explorare pe el în orele de vârf, OLTP suferă. Regula „dashboardul citește doar usage_daily” te apără; impune-o tehnic (rol read-only fără grant pe analytics_events raw, doar pe vederi/rollup).
8. **Embedding la lookup în semantic_cache** — un apel OpenAI per mesaj ne-alias la scară = latență+cost. Fix: exact-match Redis pe phrase_norm înainte; batch embeddings unde se poate.
9. **HNSW global per tabel** la tenanți foarte inegali (un client cu 200k produse lângă unul cu 300) — recall/latency per tenant mic poate degrada. Fix: ef_search tuning → partiționare pe hash(business_id) → index per partiție.
10. **Rate limits OpenAI per organizație** — la 1.000+ clienți, TPM-ul organizației devine resursă partajată; cost guard-ul per business există, adaugă și un token-bucket global per model în adapter (protejezi p95 al tuturor de spike-ul unuia).

2.4 Ce separi în servicii, ce rămâne monolit

Recomandare fermă: **monolit modular, procese multiple, un singur repo** — exact ce ai. „Microserviciu” devine doar ceea ce are *motiv operațional* propriu (scalare independentă, blast-radius, ciclu de deploy diferit):

Componentă	Verdict	Motiv
Webhook ingress	Proces separat (deja e)	Profil I/O, trebuie să rămână viu chiar când workerii suferă; scalează separat.
Worker (conversation engine)	Proces separat, N instanțe	Aici e CPU+LLM latency; HPA pe lag-ul cozii. NU îl sparge pe stagii — stagiile sunt funcții, nu servicii.
Dispatcher (outbox→Meta)	Proces separat, N instanțe	Izolează rate-limits Meta și retry-urile de restul; fairness queues.
Proactive scheduler	Proces separat, 1 instanță (leader)	Cron-like; idempotența per job o ai în design.
Catalog sync + embed + enrichment	Proces separat (batch)	Vârfuri mari, programabile noaptea, Batch API – 50%.
Dashboard/API admin	Serviciu separat (P2)	Alt ciclu de release, alți utilizatori, altă suprafață de atac.
Web widget gateway (SSE/WS)	Serviciu separat (nou)	Conexiuni persistente = profil total diferit de FastAPI request/response.
Triaj / validator / context builder	RĂMÂN în worker	Niciun beneficiu din rețea între ele; latența și simplitatea câștigă.
„Model gateway” multi-provider	Bibliotecă în adapter, NU serviciu	Un serviciu LLM-proxy separat adaugă un hop și un SPOF; failover+circuit breaker trăiesc bine în <code>llm_adapter</code> . Devine serviciu doar dacă vrei quota management central la zeci de servicii consumatoare — nu e cazul.

2.5 Probleme previzibile de performanță, cost, mentenanță

Performanță. p95 al unui tur cu agent = debounce (2–3s, prefetch ascunde o parte) + triaj nano (~0,3–0,8s) + agent mini cu 1–3 tool calls (1,5–4s) + validator (~0) + send. Ținta realistă: **p95 ≤ 6s** după debounce (exact pragul din T147). Dușmanii p95: cold start pe conexiuni DB, tool-uri care fac N+1 (compare_products să ia produsele într-un singur

query), retry-ul validatorului (dublu LLM — monitorizează rata de retry ca metrică de sănătate a promptului).

Cost. Per conversație care atinge agentul: triaj ~\$0,0002 + agent (~4-6k in din care ~70-85% cache-uit + ~400 out × 2 tururi) ≈ \$0,006-0,02 ⇒ **blended cu straturi gratuite: ~\$0,01-0,03/conversație**; embeddings neglijabil; template-uri proactive = costul Meta per mesaj (RO e în zona „rest of world”: utility tipic ~\$0,008-0,012, marketing mai scump — verifică rate card-ul curent, Meta îl poate ajusta trimestrial). Capcanele de cost: (1) re-embed accidental la sync (content_hash te apără — testează-l, T214 are DoD exact pe asta), (2) conversații-maraton fără summarizer (T087 e P1 — ok), (3) campanii marketing template la liste mari fără max-price/pacing — folosește opțiunea Meta de max-price pe marketing (nouă în 2026) când o expune API-ul tău de campanii.

Mentenanță. Cele trei locuri unde proiectul va „putrezi” dacă nu sunt păzite: (1) sincronizarea prompt ↔ taxonomie ↔ filtre tool (vocabular comun — vezi 1.3); (2) cardurile/doc-urile vs schema reală (deja s-a întâmplat o dată; instituie regula „schema_reference.md e singura autoritate de nume”); (3) golden tests care îmbătrânesc — regula „corecțiile din shadow devin golden” (T231) e antidotul, ține-o.

PARTEA 3 — Viziunea produsului și competiția (date iunie 2026)

3.1 Alinierea cu obiectivul declarat

Obiectiv: „AI Sales Assistant omnichannel pentru ecommerce”. Realitatea planului: **AI Sales Assistant pe WhatsApp pentru retail RO, livrat managed**. Diferența contează în pitch și în roadmap:

- *Sales-first*  — buying stages, mutări de vânzare decise de agent, checkout cu atribuire, proactiv comercial (coș, back-in-stock, AWB), lead scoring. Aici ești sincer cu obiectivul și diferit de 90% din piață (care e support-first).
- *Pentru ecommerce*  — catalog, variante, stoc, comenzi, curier, review intelligence; plus verticale de servicii (appointments) ca bonus.
- *Omnichannel*  azi — un canal real (WhatsApp), al doilea planificat (Telegram, P2 — canal marginal pentru retail RO), website inexistent în plan. Recomandare: redenumeste intern faza 1 „WhatsApp-first” și pune widgetul ca a doua promisiune livrabilă (V1.5), nu lăsa cuvântul „omnichannel” necontrolat în materialele de vânzare înainte de 2 august (AI Act + claims de marketing = combinație urâtă la o eventuală plângere).

3.2 Piața în 2026 — ce s-a schimbat și te avantajează

- **eMAG a educat piața.** iZi: beta în aplicație din ~26 februarie 2026, lansare oficială ~25-30 martie 2026, după soft-launch pe 50.000 de utilizatori; metrice publice: 2,6 întrebări/conversație, 1 din 3 utilizatori revin în săptămâna următoare, 5-6 pagini de produs văzute/sesiune, >90% evaluări pozitive; poate adăuga în coș și combină catalog + conversație + surse web externe (review-uri independente). Traducerea pentru tine: clientul final român *știe deja* să cumpere conversațional — tu vinzi restului pieței exact comportamentul pe care eMAG l-a validat. Și incumbenții locali se mișcă (Hornbach RO a lansat asistent AI în bricolaj) — fereastra e deschisă acum, nu la anul.
- **Prețurile competitorilor globali îți dau umbrelă de preț.** Fin: \$0,99/rezoluție (minim 50 rezoluții/lună standalone), rate de rezolvare reale raportate 42-50% (claim oficial 67% pe 40M+ conversații); Gorgias: \$0,90 (anual)/\$1,00 (lunar) per rezoluție automată, care se numără ȘI ca tichet de helpdesk (dublă taxare), automatizare reală 26-56%; Zendesk în comparațiile de piață ~\$1,50/rezoluție. Piața de AI customer service e estimată la ~\$15 mld în 2026. Costul tău marginal de \$0,01-0,03/conversație îți permite retainer fix + setup — predictibil pentru SMB-ul românesc care urăște facturile variabile.
- **WhatsApp a devenit per-mesaj (iul 2025) cu ajustări regionale în 2026** — service window gratuit nelimitat, utility gratuit în fereastră deschisă, marketing scump și cu opțiuni de max-price. Asta avantajează exact modelul tău (conversațional reactiv = aproape gratis pe canal; proactiv disciplinat pe template-uri utility).
- **GPT-5.4-mini/nano (martie 2026)** au tăiat costul exact pe profilul tău de sarcini (clasificare/extracție pe nano, vânzare pe mini, cache 10%, Batch -50%).

3.3 Tabel competitiv

	Nativx (țintă 6 luni)	Intercom Fin	Gorgias	Zendesk AI	eMA
Focus	Vânzare conversațională ecommerce	Suport (rezoluții)	Suport ecommerce (Shopify)	Suport enterprise	Shop first-j eMA
Model de preț	Retainer + setup (agenție)	\$0,99/rezoluție	\$0,90-1/rezoluție + tichete	~\$1,5/rezoluție (comparativ)	N/A (inte
WhatsApp nativ	Da, core (Cloud API direct)	Da (canal printre altele)	Limitat	Parțial	Nu

	Nativx (țintă 6 luni)	Intercom Fin	Gorgias	Zendesk AI	eMA
RO / HU / EN	Da, by design (locale în chei)	Generic multilingv	Generic	Generic	RO
Atribuire revenue în chat	Da (ref→orders→usage_daily)	Parțial (rezoluții, nu vânzări)	Parțial	Nu	Inter.
Proactiv comercial (coș/BIS/AWB)	Da, sub reguli Meta	Add-on plătit	Parțial	Parțial	N/A
Anti-halucinație structural (validator preț/link)	Da	Politici interne, opac	Opac	Opac	Inter.
Web widget	Lipsește (gap)	Da	Da	Da	În ap
Self-serve onboarding	Nu (managed)	Da	Da	Da	N/A
Voice / email / IG / FB	Nu	Da (incl. voice)	Da (email/SMS)	Da	Nu
Track record / scară	0 clienți live	40M+ conversații	15.000+ branduri	Enterprise instalat	50k+ utiliz la soft-l

3.4 Unde ești PESTE competiție (sustenabil, nu doar azi)

- Sales pe WhatsApp în limba pieței** — Fin/Gorgias tratează WhatsApp ca încă un inbox de suport; tu construiești *vânzător*, cu buying stages, comparare, checkout și follow-up nativ pe regulile Meta. Pe RO/HU nimeni global nu va investi curând în calitatea asta de limbă+verticală.
- Încredere structurală** — „zero prețuri inventate, prin construcție” e un claim demonstrabil (validator + golden tests + eval gate). În 2026, după doi ani de halucinații celebre, asta vinde la fel de bine ca feature-urile.
- Economia** — marjă de 30-90× față de pricing-ul per-rezoluție al pieței; poți vinde retainer fix unde competiția sperie SMB-ul cu facturi variabile.

4. **Bucula de bani măsurată** — „botul a generat X RON luna asta" în raportul lunar (T196) e argumentul de reînnoire pe care helpdesk-urile nu-l au.
5. **Serviciu managed cu taxonomie per vertical** — pentru SMB RO, „noi îl instalăm, îl învățăm și îl păzim" bate „self-serve cu documentație în engleză".

3.5 Unde ești SUB competiție (și ce faci)

1. **Acoperire de canale** — fără web widget, email, Messenger/Instagram, voice. Plan: widget V1.5; Messenger+Instagram în V2 (aceeași familie Meta Graph, efort mediu, păstrezi pipeline-ul); email și voice — nu în 12 luni, nu te dilua.
2. **Fără ecosistem de integrări** — Fin se pune peste orice helpdesk; tu n-ai conectori. Plan: conectorii de catalog/comenzi (Shopify, WooCommerce, feed) sunt prioritari (E14); CRM = export + webhook generic în V2, nu integrare adâncă.
3. **Maturitate eval** — Fin învață din 40M conversații; tu pornești de la zero. Antidotul tău e deja în plan: shadow mode + corecții→golden + LLM-as-judge — execută-le devreme la clientul 1, nu în P2 târziu.
4. **Self-serve & time-to-value** — onboardingul tău cere workshop de taxonomie (T216, 4h) + verificare Meta (zile). Asumă-ți: ținta nu e self-serve în 12 luni, e *onboarding managed* în ≤10 zile lucrătoare ca SLA intern.
5. **Brand/conformitate enterprise** — fără SOC2/ISO, fără referințe. Plan: nu vinzi enterprise în 12 luni; strânge 5-10 logo-uri SMB cu cifre de revenue atribuit.

3.6 Riscuri de produs

- **Platformă:** dependență dublă Meta (canal) + OpenAI (creier). Mitigări: arhitectura de canal e deja abstractă; failover LLM (NX-12, Partea 9) în primele 6 luni.
- **Incumbenți:** platformele de ecommerce locale (Gomag, Merchant Pro, Shopify) pot lansa asistenți nativi de site — încă un motiv pentru care WhatsApp + proactiv + atribuire (lucruri pe care platforma de site nu le face bine) trebuie să rămână centrul tău de greutate.
- **Reglementare:** AI Act art. 50 (2 aug 2026) — disclosure obligatoriu; GDPR pe mesagerie (consent proactiv, retenție, erase) — toate au taskuri; mută-le în categoria „blocant", nu „important".
- **Concentrare pe agenție:** modelul managed îți limitează creșterea la orele voastre. Acceptabil pentru 2026; dashboard-ul self-service din P2 e prima supapă.

PARTEA 4 — Roadmap funcțional (MVP → V3)

Notare: Impact business (★-★★★), Dificultate (▲-▲▲▲), Prioritate (P0 blocant / P1 / P2), Efort (om-zile realiste, S+J cumulat).

MVP — „un magazin demo vinde pe WhatsApp" (≈ săpt. 1-3, taskurile tale Z1-Z14 ± buffer)

Funcționalitate	Impact	Dif.	Prio	Efort	Observații
Pipeline e2e (webhook POST→...→dispatcher)	★★★	▲▲	P0	12-15 z	M1+M2 din planul tău; conține validatorul și outbox-ul — netăiabile.
search/get/compare/check_order/checkout tools	★★★	▲▲	P0	6-8 z	checkout fără product_url real → folosește URL sintetic pe demo, marcat.
FAQ aliases RO + clarify	★★	▲	P0	2 z	Limba RO hardcodată în MVP — decizie corectă, deja luată.
Golden tests + LLM replay + tenant isolation tests	★★★	▲	P0	3 z	Poarta de calitate care îți permite viteză după.
Disclosure AI (T180 → mutat în MVP)	★★★ (legal)	▲	P0	0,5 z	Art. 50(1), termen 2 aug 2026.
Cloudflare edge + Redis AOF + alerte lag/outbox (NX-01/02/03)	★★	▲	P0	1,5 z	Transformă VPS-ul din demo în producție minimă.
Atribuire simulată (orders stub + ref match)	★★★	▲	P1	2 z	Demo-ul de vânzare a serviciului depinde de cifra asta.

V1 — „clientul 1 live și profitabil" (≈ luna 2-3; P1 din planul tău + adăugiri)

Funcționalitate	Impact	Dif.	Prio	Efort	Observații
Connector catalog client 1 + quality alerts + enrichment ai_summary	★★★★	▲▲	P0	6-9 z	E14; enrichment LLM pe Batch = parte din onboarding, nu opțional.
Proactiv complet (guard consent+24h, templates, AWB, BIS, coș, STOP)	★★★★	▲▲	P0	7-9 z	E13; sub pricing per-mesaj, raportează costul template în usage_daily.
GDPR operațional (erase e2e, export, retenție, DPA)	★★★★ (legal)	▲	P0	4 z	E10; obligatoriu înainte de date reale — deja marcat așa.
Cost guard + rollup + raport lunar client	★★★★	▲	P0	4 z	T176/T195/T196 — factura retainerului.
Limbi HU/EN (detect + FAQ/clarify + golden per limbă)	★★	▲	P1	4 z	Târgu Mureș/Transilvania: HU nu e nice-to-have pentru clienții tăi.
Shadow mode + inbox-lite prin Slack (NX-08)	★★★★	▲	P0	2 z	T253 cere un mecanism de aprobare; UI-ul P2 e prea târziu — fă-l pe Slack.
Load test 50 conv + chaos kill-worker	★★	▲	P1	2 z	T147/T149 înainte de go-live, nu după.
Failover LLM (provider 2 + circuit breaker în adapter) (NX-12)	★★	▲▲	P1	3 z	P4 din Optimization Layer, acum cu task.
Monitor quality rating + tiers WhatsApp (NX-05)	★★	▲	P1	1,5 z	Protejează numărul clientului — activ, nu reactiv.

V2 — „de la 1 client la 10-30, produs vizibil" (≈ luna 4-7)

Funcționalitate	Impact	Dif.	Prio	Efort	Observații
Web widget (canal nou complet) (NX-20...26)	★★★★	▲▲	P0	12-16 z	Detaliat în Partea 7.3 — împlinește „omnichannel".

Funcționalitate	Impact	Dif.	Prio	Efort	Observații
Dashboard client v1 (analytics+revenue, inbox handoff, aprobare aliase, settings)	★★★	▲▲▲	P0	18-25 z	E15; ordinea: analytics → inbox → aprobare → settings.
Eval pipeline (judge nocturn, eval gate CI, canary 5%)	★★★	▲▲	P1	8-10 z	E16; îți permite să schimbi prompturi fără frică = viteză.
Media: STT vocale, vision foto→catalog, storage TTL	★★	▲▲	P1	8-10 z	E18; vocalele sunt mari pe WhatsApp RO — STT primul.
Messenger + Instagram DM (aceeași familie Meta)	★★	▲▲	P1	6-8 z	Reutilizezi semnătura/webhook/identities; ferestre 24h similare.
Review intelligence (sumarizare pe Batch)	★★	▲	P2	3 z	T215 — hrănește compare_products cu pros/cons reali.
CTWA referral → atribuire sursă (NX-06)	★★	▲	P2	2 z	Leagă ad spend-ul clientului de conversațiile botului.
Discount/promo logic controlat (NX-30)	★★	▲▲	P1	4-5 z	Vezi Partea 5 — azi gap; vânzătorul fără voucher e șchiop.
Export CRM generic (webhook + CSV/HubSpot light) (NX-31)	★★	▲	P2	3 z	Lead-urile calificate trebuie să ajungă undeva la client.

V3 — „enterprise & platformă" (≈ luna 8-18, selectiv)

Funcționalitate	Impact	Dif.	Prio	Efort
Multi-nod + autoscale (K8s/HPA) + Redis HA managed	★★	▲▲	P1	8-12 z
CDC → ClickHouse + explorare analytics	★★	▲▲	P2	8-10 z
Self-serve onboarding (wizard catalog+FAQ+număr)	★★★★	▲▲▲	P1	20+ z
Billing automat (Stripe: abonamente, metered pe usage_daily)	★★	▲▲	P1	6-8 z
RBAC complet dashboard + SSO (enterprise)	★★	▲▲	P2	8 z

Funcționalitate	Impact	Dif.	Prio	Efort
Rezidență date/regional LLM endpoints + DPA enterprise	★★	▲	P2	3 z
White-label / API parteneri (agenții terțe revând)	★★★★	▲▲▲	P2	20+ z
SOC2 readiness (proces, nu doar tooling)	★★	▲▲▲	P2	continuu

PARTEA 5 — Capabilități de Sales Assistant AI (checklist cerut)

Legendă status:  implementat & testat ·  proiectat (schemă/task există, cod nu) ·  absent din design.

Capabilitate	Status	E corect gândit?	Ce trebuie îmbunătățit / adăugat
RAG	●	Da — hibrid corect: filtre SQL dure + ranking semantic pgvector + reranker; FAQ și semantic_cache cu <code>locale</code> în cheie (principiul 11).	Blocat de embeddings=0 (T017→T110). Adaugă în retrieval și <code>product_sections</code> / <code>ingredients</code> pentru întrebări de compoziție (beauty le va primi). Sursa de adevăr a vocabularului de filtre (taxonomie) — vezi 1.3.
Product Catalog Retrieval	✓ (SQL) / ● (semantic)	Da — parametrizat, cap 6×8 câmpuri, preț din variantă (bug prins de T037 — semn bun de proces).	product_url NULL pe demo; la sync real, validare link viu în quality alerts (HEAD request sample).
Semantic Search	●	Da.	Prag de similaritate per limbă (RO diacritice/HU aglutinant se comportă diferit pe text-embedding-3-small) — calibrează pe T114 cu query-uri reale per limbă, nu un prag global.
Recommendation Engine	✗ (dincolo de retrieval)	Parțial — „recomandare” azi = căutare relevantă, suficient pt. V1.	V2: reguli ieftine înainte de ML: (1) cross-sell din co-achiziții (<code>order_items</code> self-join), (2) „suitable_for + concern” complement map per vertical, (3) reorder pe consumabile cu interval mediu (ai <code>reorder</code> tool — hrănește-l cu cadența reală). Fără collaborative filtering până n-ai date — nu inventa.
Personalization	●	Da — profil compact în context, locale sticky, buying stage în state.	Definește schema <code>contacts.profile</code> (chei permise per vertical) ca să nu devină depozit de gunoaie LLM; extractorul (T102) să scrie DOAR chei din whitelist.
Session Memory	●	Da — state ≤8KB cu ref-uri, istoric 8, optimistic lock.	Regula de expirare a state-ului (coș vechi de 30 zile mai e relevant?) — TTL pe chei de state în context builder.

Capabilitate	Status	E corect gândit?	Ce trebuie îmbunătățit / adăugat
Long-Term Memory	●	Da — <code>contacts.profile</code> + <code>conversation_summaries</code> + câmp RFM rezervat.	Sumarizatorul (T087) e P1 — corect; adaugă „memorie de relație” minimă în profil: ultimele 3 produse cumpărate + mărimi/nuanțe confirmate (aur curat pentru beauty).
Customer Profile	●	Da.	GDPR: profilul intră în export (T178) și în erase (deja, profile='{ }')  .
CRM Integration	✗	—	V2: export generic (webhook <code>lead.qualified</code>) + CSV zilnic) înainte de integrări native; clienții tăi SMB folosesc des doar Excel + email — nu supra-construi.
Cart Recovery	●	Da — T205 cu guard consent+24h/template.	Definește „abandon”: coș în state + 24h inactivitate + fără comandă atribuită; exclude contactele cu handoff activ.
Upsell	● (prin prompt)	Parțial — „mutarea de vânzare” e la agent, dar fără date de suport.	Dă agentului semnal explicit în context: <code>cart_value</code> , prag de transport gratuit al clientului („mai adaugă 27 lei pentru livrare gratuită” = cel mai bănos upsell din ecommerce-ul românesc).
Cross-sell	●	Ca mai sus.	Complement map per categorie în taxonomie (fond de ten → burete/primer) — determinist, nu LLM.
Discount Logic	✗	Gap real. Schema are <code>sale_price</code> , dar zero noțiune de cupoane/praguri/campanii. Vânzătorul care nu poate spune „am un cod de 10% pentru tine” pierde exact closing-ul.	NX-30: tabel <code>promotions</code> per business (cod, tip, condiții, valabilitate, buget), tool <code>get_active_promotions</code> read-only, regulă dură în validator: orice reducere menționată trebuie să existe în promotions — niciodată discount inventat sau „negociat” de

Capabilitate	Status	E corect gândit?	Ce trebuie îmbunătățit / adăugat
			LLM. Emiterea de coduri unice = doar V3, cu plafoane.
Lead Qualification	●	Da — lead_score pe semnale (stage, checkout, frecvență) + lifecycle.	Pragul de „lead fierbinte → alertă om” (P5 în Optimization Layer) să fie per business în settings; falsele pozitive ucid încrederea operatorului.
Follow-up Automation	●	Da — proactive_jobs + template approved + opt-in, exact regulile Meta 2026.	Pacing per contact (max N proactive/săptămână per contact, global peste tipuri) — azi limita e doar per tip de job.
Customer Journey Tracking	●	Da — events generice + stages + atribuire.	Lipsește doar vizualizarea (funnel în Metabase) + definiția scrisă a <code>assisted</code> (vezi 1.11).

Concluzia Părții 5: ai proiectat corect ~80% din ce definește un sales assistant performant; gap-urile reale sunt trei: **discount logic, recommendation rules ieftine, CRM export** — toate trei rezolvabile determinist, fără ML, în V2.

PARTEA 6 — Multi-tenant SaaS

6.1 Verdict pe cerințe

Cerință	Stare	Comentariu
Tenant isolation	✓ dovedit	Dublu strat: <code>WHERE business_id=\$1</code> în cod (primar) + RLS pe <code>bot_runtime</code> cu <code>app.business_id</code> (plasă). Testat live: query greșit → 0 rânduri, nu datele altuia. Test permanent T145 pe toate funcțiile din <code>db/queries</code> . Peste media industriei la stadiul ăsta.
Data isolation (PII)	✓ design	PII doar în <code>channel_identities</code> (+hash), loguri cu redaction, analytics fără PII, erase=anonimizare.
Security per tenant	●	credentials_ref → secret manager (de ales concret); token Meta per channel; lipsesc: rotație documentată, audit la citirea secretelor.

Cerință	Stare	Comentariu
Billing	● manual	Corect pentru agenție: usage_daily = sursa facturii, raport lunar T196. Stripe = V3; nu-l construi înainte de 10+ clienți.
Subscriptions	✗	<code>businesses.status</code> există (suspend posibil), dar fără noțiune de plan/limite per plan. Adaugă <code>plan</code> + limitele derivate în settings când apare al 3-lea tip de contract, nu înainte.
Permissions / RBAC	●	business_users.role fără semantică; definește owner/agent/viewer la T221 și pune-l în politicile RLS de dashboard. Lipsește separat: rol „platform admin” (voi) cu audit pe orice acțiune cross-tenant.
Quotas	✓ design	daily_cost_cap_usd per business + cost guard Redis (T176) + plafoane în settings. Adaugă quota pe template-uri proactive/zi (protecție de reputație a numărului, nu doar de cost).
Rate limits	●	Per user (token bucket per business+contact, T052) ✓; lipsesc: per-tenant la nivel de worker (fairness — un tenant în furtună nu monopolizează consumerii) și global la edge (Cloudflare).

6.2 Va ține la sute / mii de magazine?

Sute: da, cu arhitectura curentă — shared schema + business_id + RLS e modelul standard pentru SaaS B2B la scara asta; partiționarea lunară pe tabelele hot și usage_daily pentru citiri îl țin sănătos. Punctele de operare: onboarding scriptat (azi seed.ts manual → fă un `create_tenant.py` idempotent: business, channel, rol, plafoane, seed FAQ), și fairness în worker (round-robin pe conversații e implicit prin stream-per-conversație — suficient; adaugă doar limită de tururi concurente per tenant).

Mii: da, cu evoluțiile din Partea 2 (workers autoscale, Redis HA, read replica, ClickHouse). Schimbarea conceptuală necesară e una singură: **tratează „celula” ca unitate de proiectare de la ~1.500-2.000 tenanți** — nu pentru că Postgres moare, ci pentru blast-radius (un incident să afecteze 1/N din clienți) și pentru migrări fără ferestre globale. Nu construi celule acum; doar nu introduce nimic care presupune „o singură bază globală pentru totdeauna” (ex.: ID-uri secvențiale cross-tenant, joins cross-tenant în features).

6.3 Arhitectura SaaS ideală (ținta, fără date noi de construit azi)

Control plane (global, mic): registry tenanți → celulă, billing (Stripe + usage_daily agregat), onboarding, template-uri/prompturi master versionate, status page. Data plane (per celulă): exact stack-ul tău de azi (Postgres+RLS, Redis/Kafka, workers, dispatcher, scheduler). Dashboard-ul vorbește cu control plane pentru identitate și cu celula tenant-ului pentru

date. Migrarea unui tenant între celule = export/import pe business_id (încă un motiv pentru disciplina business_id pe tot).

PARTEA 7 — WhatsApp + Web Widget

7.1 WhatsApp inbound/outbound — probleme și îmbunătățiri (dincolo de ce e deja corect în design)

1. **Quality rating & messaging tiers (NX-05).** Numerele urcă/coboară tier-ele de mesaje business-initiated pe baza calității; ascultă webhook-urile de update calitate, expune în dashboard, oprește automat campaniile proactive ale unui tenant când ratingul scade — protejezi activul (numărul) clientului.
2. **Pacing pe proactiv.** Coadă separată cu rate configurabil per tenant (ex. 1 msg/s implicit) + fereastră de liniște nocturnă per timezone business (ai `timezone` în businesses — folosește-l; mesaj comercial la 3 dimineața = STOP garantat).
3. **Ferestre gratuite exploatate complet.** Utility în fereastră deschisă = gratuit → AWB-ul trimis ca răspuns la „unde e comanda?” nu costă; scheduler-ul tău verifică deja în `in_24h_window` înainte de a alege template — perfect; loghează în events care ale s-a ales (free vs paid) pentru raportul de cost.
4. **CTWA (NX-06).** Payload-ul `referral` din mesajele venite din reclame → `analytics_events(source=ctwa, ad_id)` → atribuire pe campanie; plus fereastra extinsă gratuită le face canalul de achiziție ideal pentru clienții tăi — un slide în pitch.
5. **Handoff operator: fluxul de azi e jumătate.** `request_human` setează `handoff_until` + notifică pe Slack/WhatsApp (T118) — dar operatorul răspunde de pe telefonul business? Atunci răspunsul lui NU trece prin outbox (încalcă principiul 5) și nu apare în messages decât ca echo webhook. Acceptă explicit excepția pentru P1 (mesajele operatorului intră ca `author=human_agent` din webhook echo — verifică faptul că primești echo la mesajele trimise din aplicația WhatsApp Business legată de același număr; dacă numărul e DOAR pe Cloud API, operatorul nu are aplicație — atunci inbox-lite pe Slack cu comandă de reply prin outbox devine NECESAR, nu opțional → NX-08 mutat în V1).
6. **Statusuri → retry inteligent.** `failed` cu eroare de tip „re-engagement” (în afara ferestrei) → reîncearcă automat ca template approved dacă jobul e proactiv; azi dispatcherul tratează doar retry generic.

7.2 Identificare client & analytics pe canale

`channel_identities` cu `UNIQUE(business, kind, external_id) + hash` = corect. Adaugă regula de **merge**: când același om apare pe al 2-lea canal (telefon WhatsApp == telefon

introdus în widget), unirea contactelor trebuie să fie operație auditată (audit_log) și reversibilă (păstrezi mapping-ul vechi) — merge greșit = profilul altcuiva în contextul agentului = incident de privacy. Pune merge-ul sub aprobare manuală în dashboard la început.

7.3 Web widget — arhitectura propusă (epicul lipsă, NX-20...26)

Principiu: **widgetul e un canal nou pe aceeași conductă, nu un produs nou.**

```
Browser client final
└─ widget.js (script tag, ~15KB, shadow DOM, tema per tenant)
    | HTTPS + SSE (server→client) / POST (client→server)
    ▼
Widget Gateway (serviciu nou, FastAPI/Starlette SSE)
  • emite visitor_id (cookie 1st-party) + sesiune semnată
  • rate limit per IP+visitor · CORS pe domeniile tenantului · CSP
  • inbound → ACELAȘI Redis stream inbound:{conversation_id}
  • ascultă outbox dispatch pt. canal 'web' → push SSE către browser
  ▼
Pipeline existent NESCHIMBAT (gates→...→sender→outbox)
  • channels.kind='web' · channel_identities(external_id=visitor_id)
  • FĂRĂ fereastră 24h, FĂRĂ template-uri → proactiv pe web = doar
    în sesiune activă (nudge), restul rămâne pe WhatsApp/email
  • capturare telefon în conversație → identity merge (7.2) →
    continuitate web→WhatsApp („îți trimit linkul pe WhatsApp?”)
```

Diferențe de comportament de configurat per canal (în gates/sender, pe `channel.kind`):
typing indicator = eveniment SSE; split de mesaje mai puțin agresiv; debounce mai scurt (oamenii scriu altfel pe desktop); context suplimentar = pagina curentă (`page_url`) în payload → agentul știe la ce produs se uită clientul — cel mai valoros semnal de vânzare de pe web, gratuit). Securitate specifică: domeniu de embed allowlistat per tenant, token de widget public ≠ secrete, mesajele vizitatorului tratate ca untrusted (deja sunt), și NU expune ID-uri interne în DOM.

Efort realist: 12-16 om-zile pentru v1 demo-abil (gateway + widget minimal + identitate + 2 golden tests pe canal web). Câștig: demo instant la prospecti (script tag azi, conversație în 5 minute) — scurtează ciclul de vânzare mai mult decât orice feature din P2.

PARTEA 8 — Audit de securitate

8.1 Ce e deja bine (rar la stadiul ăsta)

Semnătură webhook HMAC constant-time + dedupe + 200-la-orice (anti retry-storm); RLS testat + rol fără bypassrls + GUC per conexiune; PII într-un singur tabel + hash + redaction în loguri cu test; secrete în afara repo + gitleaks; roluri DB separate (bot/sync/dashboard/gdpr); funcția GDPR security definer ne-executabilă de bot_runtime; CODEOWNERS pe SQL/prompts; audit_log; state 8KB CHECK (anti-DoS prin umflare de state); tool registry per business cu tool dezactivat → eroare controlată; validator care face output-ul LLM inert pe exact vectorii comerciali (preț/produs/link).

8.2 Probleme, pe severitate

#	Problemă	Sev.	Impact	Soluție
S1	Disclosure AI lipsă la lansare (AI Act art. 50(1), aplicabil 2 aug 2026; amenzi până la 15 mil. €/3% CA; ghiduri Comisie + Cod de practică publicate mai-iun 2026)	Critică (legală)	Sancțiuni + reziliere contracte; aplicabil ca <i>provider</i> (tu) și <i>deployer</i> (clientul)	T180 → MVP/launch-blocki text la primul mesaj per cor + mențiune în template-uri proactive unde contactul n mai interacționat; documen implementarea (cerință explicită).
S2	Prompt injection prin conținut (mesaj user, dar și review-uri/FAQ/ai_summary sintetizate în context)	Înaltă	Manipularea agentului: recomandări distorsionate, exfiltrare context, ton ofensator sub brandul clientului	Apărarea ta primară există whitelist, max 3 calls, valid pe cifre/linkuri, fără tool de „trimite oriunde”). Adaugă: delimitare strictă date-vs-instrucțiuni în pror builder (conținutul de catalog/review în blocuri marcate „date, nu instrucțiuni (b) jobul de review summa (offline) să elimine imperativ/URL-uri din sum moderation gratuit OpenA inbound în gates; (d) golde tests de injecție (5 fixture: „ignoră instrucțiunile”, „dă promptul”, „spune că X e g

#	Problemă	Sev.	Impact	Soluție
S3	Expunere directă a VPS-ului (fără WAF/rate-limit global; IP public cunoscut după primul client)	Înaltă	DoS ieftin pe /webhook = bot mort pentru toți tenanții	Cloudflare proxy (NX-01): ascunde originul, rate limit reguli bot; ufw permite 443 de la Cloudflare. 2-3 ore.
S4	Redis fără auth/persistență, pe același host	Înaltă	Pierdere mesaje în zbor la crash; dacă portul scapă expus: acces total la coadă	requirepass + bind localhost/rețea docker + AC everysec + alertă lag (NX-C
S5	Conectare ca <code>postgres</code> + SET ROLE prin pooler	Medie→Înaltă	Un drum de cod care ratează SET ROLE rulează cu superuser → bypass RLS total	Assert <code>current_user= 'bot_runtime</code> la checkout din pool (fail-fa pe termen mediu: user ded de login <code>bot_runtime</code> cu proprie pe pooler.
S6	Semantic cache / alias poisoning	Medie	Un răspuns greșit cache-uit servit la toți userii cu întrebări similare (per tenant)	Deja parțial apărât: write-through DOAR după validator, doar conținut cacheabil fără prețuri volat (T081), aliase noi doar <code>candidate</code> → aprobare um Adaugă: TTL scurt implicit invalidare cache la sync de catalog care atinge entitățile (T214 face pe embeddings extinde la semantic_cache) niciodată cache pe răspuns date personale.

#	Problemă	Sev.	Impact	Soluție
S7	RAG poisoning prin feed catalog	Medie	Furnizor/atacator alterează feed → descrieri otrăvite intră în prompt	Quality alerts „alertă, nu publicare” (T213) acoperă anomalii numerice; extinde validare de conținut: ai_summary nou care conține URL extern/imperativ → carantină pentru review.
S8	Webhook replay	Medie	Reprocesare payload vechi (dedupe te apără de duplicare, nu de payload reconstruit)	T183 (timestamp >5min → — e P2; acceptabil, semnătura+dedupe acoperă riscul principal; urcă-l dacă trafic suspect.
S9	Endpoint admin (gdpr, toggles) cu protecție nedefinită	Medie	Acțiuni sensibile fără autz clară	Token de serviciu + IP allov audit_log pe fiecare apel (T cere „protejat” — definește standardul acum).
S10	Rezidență date / procesatori	Medie (contractuală)	Clienți RO/UE cer date în UE; OpenAI = transfer către procesator	Supabase proiect în regiun (verifică acum, mutarea ulterioară doare); DPA-uri: tu↔client (T181), tu↔Operațiune regională processin unde justifică — uplift ~10 ⁶ nano), tu↔Supabase/Meta listate în nota de informare
S11	Dependențe & supply chain	Medie	requirements pin-uit există; lipsesc scanări	pip-audit + dependabot/rei în CI; imagini cu digest pin compose prod.
S12	Merge greșit de identități	Scăzută→Medie	Profilul unui contact ajunge în contextul altuia	Merge manual/auditat la în (vezi 7.2).
S13	Jailbreak „social” (clientul cere botului opinii/medical/legal în beauty)	Scăzută	Reputațional	Risc gates (T074) + interdic persona (T131) + handoff p medical — deja în plan; ada fixture golden.

8.3 OWASP mapping rapid

A01 Broken Access Control → RLS+teste  / RBAC dashboard  (S9, T221). A02 Crypto → TLS Caddy , secrete manager  (alege unealta). A03 Injection → SQL parametrizat  verificat; prompt injection = S2. A04 Insecure Design → opusul: designul e punctul forte. A05 Misconfig → S3/S4 + verifică SSL CERT_NONE să nu scape în prod (există deja ca risc notat — pune assert pe env). A07 AuthN → Supabase Auth dashboard, webhook HMAC . A08 Integrity → CI gates, gitleaks, semnături imagini  (S11). A09 Logging → JSON+trace+redaction , alerting parțial. A10 SSRF → fără fetch arbitrar în tools (web_grounding e viitor; când vine: allowlist de domenii, deja specificat în diagramă)  design.

PARTEA 9 — Backlog nou + corecții la planul existent

9.1 Corecții la planul existent (nu taskuri noi — mutări/decizii)

1. **T180 → MVP, launch-blocking** (AI Act, 2 aug 2026). În go-live checklist: „☐ disclosure AI activ + screenshot arhivat”.
2. **T017 → azi**. Deblochează T110 (embeddings), G3 (LLM), M2.
3. **T147 + T149 → înainte de go-live clientul 1** (acum sunt P1-O1/O1 — păstrează-le acolo, dar leagă-le ca gate de T254).
4. **Carduri T021-T034: marchează OBSOLETE în fișiere** (header mare) sau regenerează-le pe schema_v2; actualizează promptul de extracție v2: „autoritatea de nume = docs/schema_reference.md; cardurile vechi NU sunt referință de schemă”.
5. **Decizie de închis săptămâna asta**: clarification_templates — tabel (recomand: da, e T036 deja scris, 1h) sau excepție documentată de la principiul 9.
6. **T118 (request_human)**: clarifică mecanismul de răspuns al operatorului (echo de la aplicația WA Business vs inbox-lite Slack) — vezi 7.1.5; de răspuns depinde dacă NX-08 e V1 sau V2.
7. **T086 (prompt caching)**: loghează `cached_tokens` în analytics_events de la primul apel — e KPI-ul tău de cost #1.

9.2 Taskuri noi (NX-*), gata de transformat în carduri

E21 • Edge & reziliență minimă de producție

ID	Task	Motiv	Prio	Est	Dep
NX-01	Cloudflare proxy în fața Caddy (DNS proxied, rate limit /webhook, allow doar CF în ufw)	S3; „WAF+API GW” din diagramă, varianta pragmatică	P0	3h	T157
NX-02	Redis: requirepass + appendonly everysec + maxmemory-policy noeviction pe stream DB	S4; mesajele în zbor sunt date	P0	2h	T048
NX-03	Alerte: consumer lag (XINFO), outbox pending depth & max age	Semnal timpuriu de saturație	P0	4h	T167
NX-04	Assert rol DB la checkout din pool (current_user=bot_runtime) + test	S5	P0	2h	T044
NX-11	systemd unit compose + restart policies + healthchecks pe toate serviciile	Repornire curată după reboot/crash	P1	3h	T156

E22 • Conformitate AI Act & GDPR operațional

ID	Task	Motiv	Prio	Est	Dep
NX-10	Disclosure AI: text per limbă, trigger „exact o dată per contact per canal”, variantă pt. template-uri proactive, doc de conformitate	S1; T180 extins la cerințele ghidurilor 2026	P0	4h	T045
NX-13	Registru procesatori + actualizare notă informare (OpenAI/Supabase/Meta/CF)	Transparență GDPR + AI Act	P1	3h	T181
NX-14	Verificare/migrare proiect Supabase în regiune UE + decizie regional processing OpenAI (cu nota de cost ~+10% nano)	S10	P1	4h	T018

E23 • WhatsApp operare avansată

ID	Task	Motiv	Prio	Est	Dep
NX-05	Quality rating & messaging-tier monitor per număr (webhook → alertă → pauză automată proactiv la rating scăzut)	Protejează numărul clientului	P1	6h	T065
NX-06	CTWA: parse referral → analytics_events(source, ad_id) + raport per campanie	Atribuire ad-spend; fereastră liberă 72h	P2	6h	T062
NX-07	Pacing proactiv per tenant + quiet hours pe businesses.timezone + cap proactiv/contact/săpt	Reputație + anti-spam	P1	6h	T201
NX-09	Retry inteligent pe failed re-engagement → template approved (doar joburi proactive)	Mai puține notificări pierdute	P2	4h	T099

E24 • Handoff operațional (înlocuiește dependența de UI-ul P2)

ID	Task	Motiv	Prio	Est	Dep
NX-08	Inbox-lite pe Slack: thread per conversație în handoff, istoric ultimele 8, comandă reply → outbox, comandă return-to-bot	T253 (shadow mode) are nevoie de mecanic real	P0 (dacă 9.1.6 o cere)	12h	T118

E25 • LLM robustețe

ID	Task	Motiv	Prio	Est	Dep
NX-12	Provider 2 în adapter (interfață comună, traducere tool schemas) + circuit breaker + flag per business	Incident OpenAI ≠ incident Nativx	P1	16h	T101
NX-15	Moderation gate inbound (endpoint gratuit) + event	S2(c)	P1	3h	T101
NX-16	Golden: 5 fixture de prompt-injection + 3 de jailbreak social	S2(d)/S13	P1	4h	T140
NX-17	schema_version în payload-urile din Redis stream + toleranță la versiuni N-1	Deploy-uri fără drenaj de coadă	P1	3h	T049

E26 • Web Widget (canalul 2 real) — vezi 7.3

ID	Task	Prio	Est
NX-20	Widget gateway SSE + sesiuni semnate + rate limit IP/visitor	P0(V2)	2,5z
NX-21	widget.js embeddabil (shadow DOM, temă din settings tenant, i18n RO/HU/EN)	P0(V2)	3z
NX-22	Canal 'web' în gates/sender (fără 24h, debounce scurt, typing=SSE)	P0(V2)	1,5z
NX-23	Identitate vizitator + capturare telefon → merge auditat (7.2)	P0(V2)	2z
NX-24	Context pagină (page_url/product în payload → retrieval hint)	P1(V2)	1z
NX-25	CORS/CSP/embed allowlist per tenant + token public	P0(V2)	1z
NX-26	Golden tests canal web + load test SSE 200 sesiuni	P1(V2)	1,5z

E27 • Vânzare avansată (gap-urile din Partea 5)

ID	Task	Prio	Est
NX-30	Promotions: tabel + tool read-only + regulă validator „discount ⊆ promotions"	P1(V2)	2,5z
NX-31	Export CRM: webhook lead.qualified + CSV zilnic per tenant	P2(V2)	1,5z
NX-32	Cross-sell map per categorii (taxonomie) + semnal free-shipping-gap în context	P1(V2)	2z
NX-33	Funnel + cohortă revenire în Metabase (definiția assisted scrisă)	P1(V1)	1z

E28 • Date & calitate

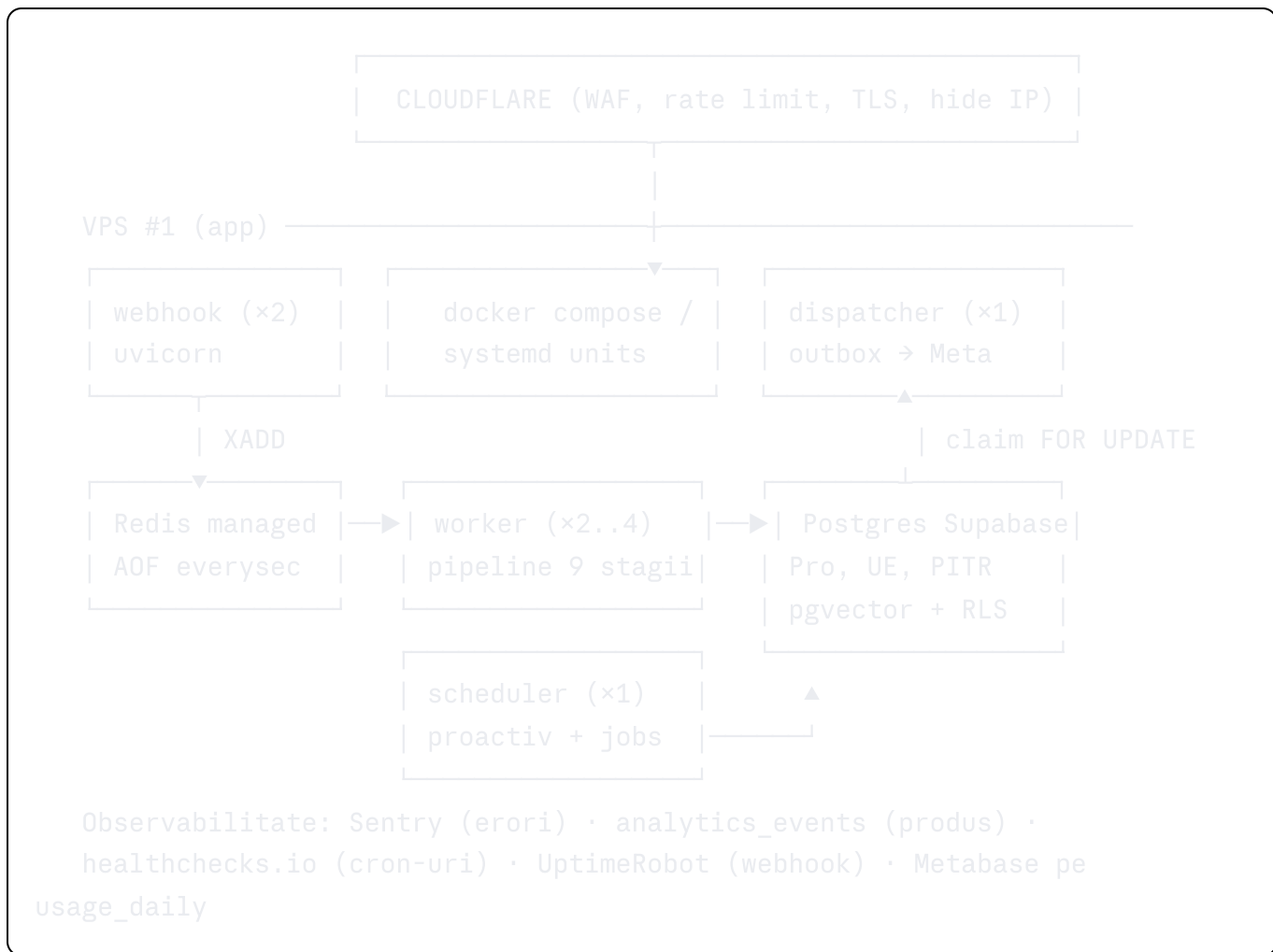
ID	Task	Prio	Est
NX-40	pg_partman/pg_cron: creare automată partiții +2 luni, alertă la lipsă	P0(V1)	3h
NX-41	Onboarding script create_tenant.py idempotent (business, channel, rol, plafoane, FAQ seed)	P1(V1)	6h
NX-42	pip-audit + renovate în CI; digest pin pe imagini prod	P1(V1)	4h
NX-43	Schemă pentru contacts.profile (whitelist chei per vertical) + enforcement în extractor	P1(V1)	4h

PARTEA 10 — Arhitectura finală recomandată (orizont 3 ani)

Teza acestei părți: **nu propun o rescriere, propun o secvențiere**. Designul existent (pipeline liniar, Postgres-first, LLM în exact două puncte, outbox, multi-tenant pe `business_id`) este corect pentru următorii 3 ani. Greșeala clasică în faza voastră este să construiești acum infrastructura de la 10.000 de clienți și să mori cu 0 clienți. Planul de mai jos leagă fiecare investiție de un **prag de business măsurabil**, nu de un calendar arbitrar. Fazele se activează când pragul e atins, nu când vine luna din calendar.

10.1 Faza A — „monolit modular pe infrastructură serioasă" (0-6 luni, 1-10 clienți)

Stack-ul rămâne cel actual: FastAPI + Redis Streams + Supabase Postgres (pgvector) + GPT-5.4 mini/nano + Meta Cloud API direct. Ce se schimbă este **modul de rulare**, nu tehnologia.



Schimbările obligatorii ale fazei A, în ordinea în care le-aș face: Cloudflare în fața webhook-ului (NX-01, jumătate de zi, elimină clasa întreagă de atacuri direct-pe-IP); Redis mutat pe managed cu AOF (everysec) sau măcar AOF activat pe VPS (NX-02 — azi un restart Redis pierde mesaje primite și neprocesate); Supabase Pro cu rezidență UE și PITR înainte de primul client plătit (S10 — pe Free nu ai backup-uri serioase și nici DPA solid); Sentry pentru excepții cu redaction de PII în logger (principiul 12 aplicat și la observabilitate); separarea proceselor — webhook, worker, dispatcher, scheduler rulează ca unități systemd/compose separate cu restart policy, NU într-un singur proces Python. Aceasta din urmă e gratuită ca efort (structura (src/) o permite deja) și e diferența dintre „a căzut agentul” și „a căzut tot”.

Deploy-ul în faza A: GitHub Actions → build imagine → push registry → (docker compose pull && up -d) pe VPS prin SSH, cu healthcheck înainte de a tăia versiunea veche. Nu Kubernetes. Nu Terraform pe un VPS. Eval gate-ul din diagrama voastră de go-live (golden tests pass = condiție de merge) intră în CI din prima zi în care există golden tests.

Pragul de ieșire din faza A: **>10 clienți activi SAU >50.000 conversații/lună SAU consumer lag care nu mai coboară sub 30s în orele de vârf.**

10.2 Faza B — „aceleași piese, redundante și măsurate” (6-18 luni, 10-100 clienți)

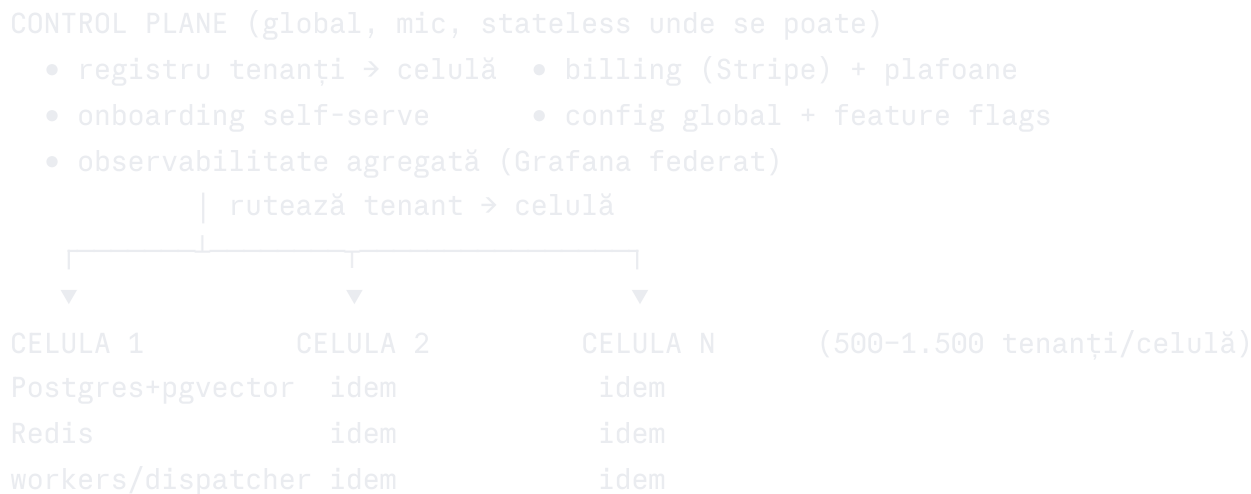
Componentă	Schimbare	Trigger explicit (nu „când avem timp”)
Compute	K8s managed (DOKS/Scaleway) sau k3s pe 3 noduri Hetzner; HPA pe workers după consumer lag	>2 VPS-uri administrate manual sau lag repetat
Redis	HA (replica + Sentinel) sau managed echivalent	primul incident de pierdere/indisponibilitate
Postgres	read replica pentru dashboard/Metabase; pgbouncer dedicat	query-uri analitice >500ms pe primary
Analytics	ClickHouse + CDC (Debezium/peerdb) din <code>analytics_events</code>	>50M evenimente/lună SAU dashboard >2s
Coadă	Kafka DOAR dacă: >100 msg/s susținut, SAU nevoie de replay istoric, SAU multi-consumer pe inbound	altfel Redis Streams rămâne — e suficient și simplu
LLM	model gateway ca bibliotecă (nu serviciu): failover OpenAI→Anthropic (NX-12), buget per tenant, circuit breaker	din prima zi a fazei B; serviciu separat abia la >8 ingineri
Canale	Widget web GA (E26) + Messenger/Instagram ca <code>channel_kind</code> nou peste <code>channel_identities</code> existent	cererea clienților
Calitate	eval pipeline în CI: golden tests obligatorii la merge, LLM-as-judge săptămânal pe eșantion din prod	de la primul client real
Dashboard	read-only pe <code>usage_daily</code> → apoi self-service FAQ/aliase/template-uri	>5 clienți care întreabă „cât a vândut botul?”

Punctul important al fazei B este ce **NU** se schimbă: pipeline-ul de 9 stagii, schema DB, contractul TurnContext, outbox-ul. Scalarea e orizontală (mai mulți workers identici), nu arhitecturală. Dacă în faza B simțiți nevoia să rescrieți pipeline-ul, înseamnă că faza A a fost greșită — semnalați-mi.

Pragul de ieșire: >100 clienți activi SAU cerințe enterprise (SLA contractual, SSO, rezidență dedicată) SAU >500.000 conversații/lună.

10.3 Faza C — „celule, nu microservicii” (18–36 luni, 100–1.000+ clienți)

La scara asta problema nu mai e throughput-ul, ci **raza de explozie** (blast radius) și onboarding-ul. Soluția corectă pentru SaaS multi-tenant la mii de tenanți este arhitectura celulară, nu descompunerea în 30 de microservicii:



Fiecare celulă e o copie completă a stack-ului din faza B. Un incident, o migrare greșită sau un tenant abuziv afectează o celulă, nu platforma. Onboarding-ul devine „alocă tenant în celula cu cel mai mic load” (NX-41 devine API-ul control plane-ului). Multi-regiune apare **doar dacă** semnați clienți non-UE — altfel e cost fără beneficiu. Vector search: pgvector ține lejer până la ~5–10M embeddings per celulă; peste, Qdrant per celulă — dar la 1.000 de magazine a câte 5.000 de produse sunteți la 5M total, deci probabil nu aveți nevoie niciodată. Self-serve complet + SOC 2 Type I → II intră aici, pentru că abia aici le cere piața.

Microserviciile reale rămân puține și pe seams deja existente: sync service (deja separat logic), model gateway (doar dacă echipa depășește ~8 ingineri și 2 echipe își dispută biblioteca), eventual un serviciu de media (STT/Vision). Stagiile pipeline-ului NU devin servicii niciodată — un hop de rețea per stagiou ar transforma p95 de 6s în 12s fără niciun câștig.

10.4 Deciziile arhitecturale, explicate (de ce, nu doar ce)

Pipeline liniar, nu agent graph/framework (LangGraph etc.). Vânzarea pe WhatsApp e un flux secvențial cu early-exit, nu un graf cu bucle. Pipeline-ul liniar e debugabil (un turn = o linie de log per stagiou), testabil pe stagii izolate și are latență predictibilă. Framework-urile de agenți aduc bucle de orchestrare, stare ascunsă și dependență de API-uri instabile — exact ce interzic principiile 1 și 3. Costul deciziei: flexibilitate mai mică la fluxuri exotice; acceptabil, pentru că CLARIFY ieftin acoperă ambiguitatea.

Postgres-first, o singură bază, schemă plată pe `business_id`. Tranzacționalitatea Sender-ului (reply + state + outbox în aceeași TX) este fundamentul garanției „exact-once spre client” — asta cere o singură bază relațională. Partiționarea pe lună la `messages`/`analytics_events` plus RLS ca plasă acoperă și volumul și izolarea. NoSQL ar fragmenta tranzacția; schemă-per-tenant ar muri operațional la 100+ tenanți (migrări ×100).

pgvector, nu vector DB dedicat. Căutarea voastră e „filtre SQL dure + ranking semantic pe rest” — exact cazul în care pgvector strălucește, pentru că filtrul și vectorul stau în același query plan. Un Qdrant/Pinecone separat ar însemna dual-write, sincronizare și un sistem în plus de operat, pentru un câștig de recall irelevant sub 5M vectori. Pragul de re-evaluare e scris la faza C.

Redis Streams, nu Kafka (încă). Aveți nevoie de FIFO per conversație + lock + debounce la <100 msg/s. Redis Streams face asta în 200 de linii de cod și 1 GB RAM. Kafka aduce rebalansări de consumer group, operare ZK/KRaft și nimic ce folosiți azi. Condițiile de trecere sunt scrise în tabelul fazei B — până atunci, AOF everysec + idempotența existentă (dedupe pe `provider_msg_id`, outbox cu `idempotency_key`) acoperă durabilitatea.

Meta Cloud API direct, nu BSP (Twilio/360dialog). Diferența e ~\$0,005-0,01/mesaj markup BSP — la 100k mesaje/lună înseamnă \$500-1.000/lună marjă pierdută, adică exact diferența voastră competitivă de cost față de Intercom/Gorgias. Costul deciziei: voi gestionați verificarea Meta, quality rating-ul și template-urile (NX-05, NX-06). Corect pentru un produs al cărui șanț competitiv e costul per conversație.

GPT-5.4 mini/nano cu failover Anthropic ca bibliotecă, nu self-hosted LLM. La \$0,01-0,03/conversație, inferența e deja sub 5% din prețul vostru de vânzare; self-hosting ar economisi nimic și ar adăuga MLOps. Failover-ul (NX-12) e necesar nu pentru cost, ci pentru SLO: un outage OpenAI de 2h fără failover înseamnă „botul tace” — încălcarea principiului 6 la nivel de platformă. Gateway-ul rămâne bibliotecă importată de worker; serviciu separat abia când mai multe aplicații îl consumă.

SSE, nu WebSocket, pentru widget. Fluxul e asimetric (vizitatorul scrie rar prin POST, serverul împinge tokens/typing). SSE trece prin orice proxy corporate, se reconectează nativ (`Last-Event-ID`) și nu cere infrastructură stateful de conexiuni. WebSocket ar adăuga complexitate pentru un câștig pe care nu-l folosiți.

Supabase rămâne, cu plan de ieșire scris. Pro UE + PITR acoperă faza A și B. Mutarea pe RDS/Cloud SQL self-managed devine subiect abia dacă: aveți nevoie de extensii necontrolabile, pooler-ul devine bottleneck măsurat, sau costul depășește echivalentul RDS cu >40%. Schema e Postgres vanilla — portabilitatea e reală, deci decizia nu e o căsătorie.

ClickHouse abia la prag, cu CDC, nu dual-write. Dual-write din worker ar încălca principiul 5 (un singur punct de ieșire) și ar introduce divergență. CDC din

`analytics_events` (care e deja append-only) e curat și reversibil.

10.5 Tabelul final al componentelor (țintă la 36 luni)

Strat	Tehnologie	Observație
Frontend clienți	widget.js (shadow DOM) + SSE	embed 1 linie, temă per tenant
Dashboard	Next.js + Supabase Auth (RLS <code>business_users</code>)	read-only → self-service
Edge	Cloudflare (WAF, rate limit, bot mgmt)	din faza A
API/Webhook	FastAPI, stateless, ×N	semnătură Meta, ACK <50ms
Backbone mesaje	Redis Streams (→ Kafka doar la praguri)	FIFO + lock per conversație
Orchestrare AI	pipeline 9 stagii, cod propriu	NU framework de agenți
Model gateway	bibliotecă: OpenAI primar, Anthropic failover, buget/tenant	circuit breaker + cache prompt
Vector	pgvector HNSW per tenant (→ Qdrant per celulă peste 5-10M)	filtru+vector în același plan
RAG	faqs + semantic_cache + product_embeddings, locale-aware	straturi gratuite înainte de LLM
Memory	conversations.state 8KB + summaries + contacts.profile	bugete impuse în cod
Recommendation	reguli (cross-sell map, free-shipping gap) + ranking semantic	nu ML dedicat sub 100k comenzi
Event bus intern	analytics_events append-only → CDC	un singur scriitor per fapt
Analytics	usage_daily (facturare) + Metabase → ClickHouse la prag	dashboard-ul nu atinge primary
Observabilitate	Sentry + structlog JSON + Grafana/Prometheus (faza B)	redaction PII în logger

Strat	Tehnologie	Observație
CI/CD	GitHub Actions: ruff+pytest+golden gate → canary 5% (faza B)	eval gate = condiție de merge
Infra	VPS×2 → K8s managed → celule	IaC abia din faza B (Terraform)
Securitate	RLS bot_runtime + secret manager + Cloudflare + moderation inbound	S1-S13 din Partea 8

Scorurile finale

Arhitectură: 8/10. Designul pe hârtie e de top: pipeline liniar cu un singur scriitor per câmp, outbox tranzacțional, multi-tenant cu RLS dovedit live, partiționare din prima zi, buget de context impus în cod, principii scrise și — rar — respectate în puținul cod existent. Minus 2 puncte pentru: golul dintre diagramele de producție (canary, RTO 1h, multi-AZ) și realitatea VPS unic + Redis nepersistent; cardurile T021-T034 contradictorii cu schema_v2 încă nemarcate obsolete în zip; absența oricărei piese pentru canalul web deși produsul e definit „omnichannel”.

Scalabilitate: 7/10. Drumul 1→100 clienți e acoperit de stack-ul actual fără modificări structurale; 100→1.000 cere doar redundanță (workers ×N, Redis HA, read replica), nu re-design; schema partiționată + `usage_daily` rollup arată că volumul a fost gândit. Minus 3 puncte pentru: single points of failure peste tot azi (VPS, Redis, dispatcher unic), dependența de un singur furnizor LLM fără failover, și pooler-ul Supabase + conectarea `postgres` + `SET ROLE` nevalidată sub load real.

Produs: 6/10. Diferențierea economică e reală și mare (cost/conversație de 30–90× sub Fin/Gorgias), focusul pe vânzare-nu-suport e corect, atribuirea prin `ref_code` închide bucla de bani — astea sunt fundații de produs bune. Minus 4 puncte pentru: zero prezență web (canalul unde stă majoritatea traficului ecommerce RO), zero dashboard pentru client (retainerul devine greu de apărat), promo/discount logic lipsă (prima cerință a oricărui magazin în sezon de reduceri), și un singur vertical demo cu date sintetice — PMF-ul e încă o ipoteză, nu un fapt.

AI capabilities: 6,5/10. Triaj ieftin + agent cu tools + validator anti-halucinație + straturi gratuite 40–60% + prompt caching = un design AI matur economic, peste media pieței la cost. Minus 3,5 puncte pentru: nimic din el nu rulează încă (embeddings=0, faqs=0, niciun apel LLM făcut), lipsesc moderation/anti-injecție (S2) și failover-ul, evaluarea (golden +

LLM-as-judge) există doar ca tabele goale, iar personalizarea reală (profil → recomandare) e schiță.

Top 20 priorități (ordinea în care le-aș executa de luni dimineață)

1. **T017 — cheia OpenAI** (azi, 30 min): deblochează embeddings, triaj, agent; e pe drumul critic al absolut tot.
2. **T180 → MVP, nu P1-O2 — disclosure AI** (2h): obligație legală la 2 aug 2026; un rând în system prompt + primul mesaj „Asistent virtual Nativx”.
3. **NX-01 — Cloudflare în fața webhook-ului** (3h): elimină atac direct pe IP, rate limit L7 gratuit.
4. **NX-02 — Redis AOF everysec + parolă + bind** (2h): azi un restart pierde mesaje clienți.
5. **G1 — queries DB** (contacts/conversations/messages/outbox) (2z): fundația oricărui mesaj procesat.
6. **G2 — webhook POST + consumer + runner → M1 echo e2e** (3z): primul mesaj WhatsApp care primește răspuns.
7. **NX-40 — pg_partman pentru partiții** (3h): fără el, în luna 3 insert-urile în `messages` crapă.
8. **T110 + embed_products — embeddings pe demo** (4h): fără ele search-ul semantic și cache-ul nu există.
9. **G3+G4 — triaj + agent + validator + sender → M2 conversație de vânzare** (5z): turul complet care vinde.
10. **NX-16 — golden tests injecție/leakage în CI** (1z): gate-ul care nu lasă regresii de siguranță pe prod.
11. **NX-12 — failover LLM OpenAI→Anthropic** (1,5z): „botul tace” nu e un mod de eșec acceptabil pentru un SLA.
12. **T016 — verificarea Meta Business pornită ACUM** (30 min azi): durează 3-15 zile calendaristice, blochează clientul 1.
13. **M3 — go-live demo pe VPS cu procese separate + Sentry** (2z): prima producție reală, cu disclosure live.
14. **NX-08 — inbox-lite handoff (Slack/echo WA Business)** (1,5z): fără răspunsul operatorului, handoff-ul e o fundătură.
15. **NX-05 — monitor quality rating + pacing template-uri** (1z): un ban Meta = produsul mort; trebuie văzut înainte.

16. **T210/T216 — onboarding client 1 real (sync catalog + product_url) (3-5z):** prima validare PMF cu bani.
 17. **Supabase Pro UE + Pitr + DPA-uri semnate (2h + legal):** înainte de primul contact real în DB (S10).
 18. **T177/T178 — gdpr_erase + export testate e2e (1z):** cerere GDPR la client real = obligație, nu feature.
 19. **Dashboard read-only pe usage_daily (Metabase shared) (1z):** clientul vede „ce a vândut botul” — apără retainerul.
 20. **Decizie scrisă: web widget ca V1.5 (NX-20...26, 12-16 om-zile) :** calendarul real al „omnichannel” — fără ea, povestea de produs rămâne WhatsApp-only.
-

Roadmap 6 luni (iunie → noiembrie 2026)

Capacitate presupusă: Senior ~10h/zi lucrătoare (~210h/lună), Junior ~4h/zi (~85h/lună). Buffer 20% inclus în formulări — dacă o lună „dă pe afară”, taie din coloana V1, niciodată din Siguranță.

Iunie (acum → 30 iun) — MVP + legal + primul metal. M1 echo e2e (Z5), M2 conversație vânzare pe demo (Z10), M3 live pe VPS (Z14) cu T180 disclosure, NX-01 Cloudflare, NX-02 Redis AOF, NX-40 partiții automate, T110 embeddings, T016 verificare Meta pornită, T017 cheie. Junior: golden tests scrise din transcripturi demo + README operare. Ieșirea lunii: un număr WhatsApp pe care oricine poate scrie și primește o conversație de vânzare conformă legal.

Iulie — hardening + client 1. NX-12 failover, NX-15 moderation inbound, NX-16 golden injecție în CI, NX-05 quality monitor, NX-08 inbox-lite, Supabase Pro UE, Sentry, Metabase intern, straturi gratuite complete (alias + semantic_cache cu TTL+invalidare NX-S6). Onboarding client 1 (T210 sync catalog, T216 product_url, FAQ reale). Ieșirea lunii: client 1 în shadow mode → live, cost/conversație măsurat real.

August — conformitate verificată + proactiv + client 2. 2 aug: audit intern AI Act (disclosure pe mesaj inițial, template-uri, widget-spec) — o zi, cu dovezi arhivate. Proactiv complet: AWB, back-in-stock, abandoned cart cu consent + template approved (T140-range). NX-41 create_tenant idempotent, NX-43 schemă profile. Client 2 (al doilea vertical — HVAC sau salon, ca să validăm prompt_builder pe alt catalog). Ieșirea lunii: 2 clienți plători, proactivul generează primele conversii atribuite.

Septembrie — V1 vizibil pentru client. Dashboard read-only (usage_daily + conversații + handoff-uri), NX-33 funnel + cohorte, NX-30 promotions (tabel + tool + regulă validator), NX-32 cross-sell map. Load test 1.000 conversații/zi sintetic pe staging. Clienți 3-5. Ieșirea

lunii: clientul își vede singur ROI-ul; agentul vinde cu promoții reale fără să inventeze discounturi.

Octombrie — widget web (start) + scară mică. NX-20 gateway SSE, NX-21 widget.js, NX-22 canal web în pipeline, NX-25 CORS/CSP. Telegram doar dacă un client plătește pentru el. Clienți 5-8; primul exercițiu de restore din backup (S8 — restore test, nu doar backup). Ieșirea lunii: widget funcțional pe staging, pe site-ul demo Nativx.

Noiembrie — widget beta + decizie de scară. NX-23 identity merge auditat, NX-24 context pagină, NX-26 golden web + load SSE. Widget beta la 1-2 clienți existenți. Retrospectivă infra: rămânem pe VPS×2 sau pornim K8s (decizie pe date: lag, incidente, ore de operare/lună). Clienți 8-10. Ieșirea semestrului: platformă pe 2 canale, 8-10 clienți, conformă AI Act, cu failover LLM și backup-uri testate — adică exact „V1” din Partea 4.

Roadmap 12 luni (decembrie 2026 → iunie 2027)

Dec 2026 - Ian 2027 — widget GA + self-service light. Widget GA cu temă per tenant și i18n complet RO/HU/EN; dashboard v2: clientul își editează singur FAQ-urile, aliasele și template-urile (cu workflow approve); facturare semi-automată din usage_daily (factură generată, plată încă manuală). Sezonul de sărbători = primul test real de volum pe abandoned cart + promoții; quality rating monitorizat zilnic (NX-05 devine alertă pe telefon).

Feb - Mar 2027 — date la scară + al treilea canal. Read replica pentru dashboard; ClickHouse + CDC doar dacă pragurile din faza B sunt atinse (altfel se amână — decizie pe numere, scrisă). Messenger/Instagram ca `channel_kind` nou peste identity-ul existent — efort mic, cerere probabilă de la clienții fashion/beauty. Eval pipeline v2: LLM-as-judge săptămânal pe eșantion prod + scor de calitate per tenant în dashboard. Țintă: 15-20 clienți; primul contractor part-time pe suport/onboarding.

Apr - Iun 2027 — self-serve beta + pregătirea scării. Stripe billing (abonament + metered pe conversații), onboarding self-serve beta (create_tenant API + wizard catalog CSV/feed), SOC 2 gap analysis (nu certificare încă — doar lista de lipsuri cu costuri), gap analysis celule (la ce număr de tenanți pornim celula 2 — probabil nu încă). Multi-vertical validat: beauty + HVAC + salon cu prompt_builder din taxonomie, fără cod per vertical. Țintă 12 luni: **20-30 clienți plătitori, 2 canale GA + 1 beta, marjă brută pe client >85%, zero incidente de izolare tenant, audit AI Act trecut.**

Surse & date de piață 2026 folosite în analiză

Prețuri și poziționare competiție: featurebase.app/blog/fin-ai-pricing și fin.ai/learn/ai-

customer-service-agent-pricing-comparison (Fin \$0,99/rezoluție, minim 50/lună, Zendesk ~\$1,50/rezoluție, rate reale de rezoluție 42-50% vs claim 67%); minami.ai/blog/intercom-fin-ai-agent-pricing (mecanica facturării per rezoluție); lindy.ai/blog/gorgias-pricing și chatarmin.com/en/blog/gorgias-pricing (Gorgias \$0,90-1,00/rezoluție automată care se numără și ca tichet, automatizare tipică 26-56%, benchmark uman ~\$3,10/tichet, 15.000+ branduri).

eMAG iZi: wall-street.ro și start-up.ro (beta în aplicație ~26 feb 2026, lansare oficială ~25-30 mar 2026, soft-launch ~50k utilizatori); nwradu.ro (comportament: ~2,6 întrebări/conversație, 1 din 3 utilizatori revin în săptămâna următoare, 5-6 pagini de produs vizitate/sesiune, >90% evaluări pozitive, control coș + grounding pe review-uri externe; Hornbach RO cu asistent propriu de bricolaj).

WhatsApp pricing: developers.facebook.com/docs/whatsapp/pricing (model per-mesaj template din 1 iul 2025, service window 24h gratuit, utility gratuit în fereastra deschisă, CTWA cu fereastră 72h gratuită, ajustări regionale + 16 valute din ian 2026, calendar tarifar trimestrial); sleekflow.io și blueticks.co (sinteze 2026, utility RoW ~\$0,008-0,012/mesaj, opțiuni max-price pe marketing).

Modele LLM: developers.openai.com/api/docs/models (GPT-5.4-mini \$0,75/\$4,50 per 1M tokens, nano \$0,20/\$1,25, cache input 10%, Batch -50%, context 400K, lansate 17 mar 2026, cutoff aug 2025); benchlm.ai (comparativ și uplift regional EU +10% pe nano).

Reglementare: artificialintelligenceact.eu/transparency-rules-article-50 (art. 50(1) — obligația de a informa utilizatorul că interacționează cu AI, aplicabilă de la 2 august 2026; amenzi până la 15 mil € / 3% din cifra de afaceri globală); globalpolicywatch.com și belto.ai (ghidurile Comisiei mai 2026 + Codul de practică din 10 iun 2026; amânarea high-risk la dec 2027 NU atinge art. 50).

Piață: estimări AI customer service software ~\$15 mld în 2026 (sinteze featurebase/minami citate mai sus). Toate cifrele de cost intern Nativx (\$0,01-0,03/conversație) sunt calculate din prețurile GPT-5.4 de mai sus pe bugetele de context din CLAUDE.md, nu preluate din surse externe.

Raport generat 12 iunie 2026 · Audit CTO Nativx Assistant · Toate ID-urile T-xxx referă planul existent (193 taskuri), ID-urile NX-xx referă backlogul nou din Partea 9.